

UNIT – II

Requirement Engineering:

**Requirement Engineering tasks,
Initiating the Requirements Engineering Process,
Eliciting Requirements**

Requirements Engg.

- Helps s/w engg's to better understand the pblm they will work to solve.
- Encompasses the set of tasks that lead to an understanding of
 - what the buss. impact of the s/w will be,
 - what the customer wants, and
 - how end-users will interact with the s/w.
- S/w engg's* (sometimes referred to as sys. engg's or analysts in the IT world) and *other prjt stakeholders* (managers, customers, end-users) all participate in requirements engg.

- It is imp. for designing and building an elegant comp. pgm that solves the wrong pblm which serves no one's needs.

A Bridge to Design and Construction

- Designing and building comp. s/w is challenging, creative, and just plain fun.

- RE, like all other s/w engg. activities, must be adapted to the needs of

- i)the process,

- ii)the prjt,

- iii)the product, and

- iv)the people doing the work.

- In perspective process models, RE is a s/w engg. action that begins during the communication activity and continues into the modeling activity.
- It is essential that the s/w team make a real effort to understand the requirements of a pblm before the team attempts to solve the pblm.
- RE builds a bridge to **design** and **construction**.

•The journey across the bridge takes us high above the prjt, allowing

1)s/w team to examine the context of the s/w work to be performed;

2)the specific needs that design and construction must address;

3)the priorities that guide the order in which work is to be completed; and

4)the info., functions, and behaviors that will have a profound impact on the resultant design.

Requirements Engineering Tasks

The RE process is accomplished through the execution of **7 distinct functions**:

- 1)inception,
- 2)elicitation,
- 3)elaboration,
- 4)negotiation,
- 5)specification,
- 6)validation, and
- 7)management.

1.Inception

•s/w engg's ask a set of context-free questions.

1.First set(focuses on cust. & other stakeholders) of context-free questions:

1)Who is behind the request for this work?

2)Who will use the solution?

3)What will be the economic benefit of a successful solution?

4)Is there another source for the solution that you need?

II. Next set of questions (enables the s/w team to gain a better understanding of the pblm):

- 1) How would you characterize "good" output that would be generated by a successful solution?**
- 2) What pblm (s) will this solution address?**
- 3) Can you show me (or describe) the buss environment in which the solution will be used?**
- 4) Will special performance issues or constraints affect the way the solution is approached?**

III.Final set of questions(focuses on the effectiveness of the communication activity itself):

**1)Are you the right person to answer these questions?
Are your answers "official"?**

2)Are my questions relevant to the problem that you have?

3)Am I asking too many questions?

4)Can anyone else provide additional information?

5)Should I be asking you anything else?

Intent:

- To establish a **basic understanding** of the **pblm**,
- The **people** who want a **solution**,
- The **nature of the solution** that is desired,
and
- The effectiveness of **preliminary communication and collaboration** b/n the customer and the developer.

2.Elicitation

It certainly seems simple enough

—ask the **customer**, the **users**, and others what the **objectives for the system or product** are,

-what is to be **accomplished**,

-how the **system or product is to be used** on a day-to-day basis.

- But it isn't simple—it's very hard.

-**Christel and Kang** identified a no. of pblms that help us understand why requirements elicitation is difficult:

i)Problems of scope:

- The **boundary** of the system is **ill-defined** or the **customers/users** specify **unnecessary technical detail** that may confuse.

ii)Problems of understanding:

- The **customers/users** are not completely **sure of what is needed**, have a poor understanding of the capabilities.
- don't have a **full understanding of the pblm** domain,

iii) Problems of volatility:

- The **requirements change** over time.
- To overcome these pblms, **REs must approach the requirements gathering activity** in an organized manner.

3.Elaboration

- The **info. obtained** from the **customer** during **inception and elicitation** is **expanded** and **refined** during “elaboration”.
- This requirements engg. activity **focuses on** developing a **refined technical model** of **s/w** **functions**, **features** and **constraints**.

4.Negotiation

- It is unusual for **customers and users** to ask for **more than can be achieved**, given **limited buss resources**.
- It is also relatively common for **diff. customers or users** to **propose conflicting requirements**, **arguing** that their version is "essential for our special needs."
- The requirements engg. must reconcile these conflicts through a process of negotiation.

- Rough "guesstimates" of development effort are made and used to assess the impact of each requirement on project cost and delivery time.

- Using an iterative approach, requirements are eliminated, combined, and/or modified so that each party achieves some measure of satisfaction.

5.Specification: In the context of s/w, the term specification means **diff. things to diff. people.**

•It can be

1)a written doc.,

2)a set of graphical models,

3)a formal mathematical model,

4)a collection of usage scenarios,

5)a prototype, or any combination of these.

•"standard template" should be developed and used for a specification.

•It describes the **function and performance** of a comp.-based sys. and the constraints that will govern its development.

6.Validation

- The work products produced as a consequence of RE are assessed for quality during a validation step.
- It examines the specification to ensure that all s/w requirements have been stated **unambiguously**, that **inconsistencies, omissions, and errors** have been detected and corrected

7.Requirements management

- It is a set of activities that help the **prjt team** identify, control, and track requirements & changes to requirements at any time as the project proceeds.
- It is similar to SCM.
- RM begins with identification.
- Each requirement is assigned a unique identifier.
- Once requirements are identified, traceability table(TT) are developed, each TT relates requirements to one or more aspects of the sys. or its environment.

The following are the diff. possible TTs:

- **Fig 7.1 generic traceability table(from TB)**

Software Requirements Specification Document (SRS)

Systems Requirements Specification

- SRS is the official statement of what the system developers should implement.
- SRS is a complete description of the behavior of the system to be developed.
- SRS should include both a definition of user requirements and a specification of the system requirements.
- The SRS fully describes what the software will do and how it will be expected to perform.

Systems Requirements Specification

Table of Contents:

- I. Introduction
- II. General Description
- III. Functional Requirements
- IV. Non Functional Requirements

1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions, acronyms, and abbreviations
 - 1.4 References
 - 1.5 Document overview
2. General description
 - 2.1 Product perspective
 - 2.2 Product functions
 - 2.3 User characteristics
 - 2.4 General constraints
 - 2.5 Assumptions and dependencies
3. Specific requirements
 - 3.1 Functional requirements
 - 3.1.1 Functional requirement 1
 - 3.1.1.1 Introduction
 - 3.1.1.2 Input
 - 3.1.1.3 Processing
 - 3.1.1.4 Output
 - 3.1.N Functional requirement M
 - 3.2 External interface requirements
 - 3.3 Performance requirements
 - 3.4 Design constraints
 - 3.5 Security requirements
 - 3.6 Maintainability requirements
 - 3.7 Reliability requirements
 - 3.8 Availability requirements
 - 3.9 Database requirements
 - 3.10 Documentation requirements
 - 3.11 Safety requirements
 - 3.12 Operational requirements
 - 3.13 Site adaptation

Systems Requirements Specification

1.INTRODUCTION

- **Purpose**
 - ✓ Describe the purpose of the SRS, not the purpose of the software being developed.
 - ✓ Intended audience for SRS.
- **Scope**
 - ✓ Describe application of software (benefits, objectives).
 - ✓ Explain what software will (not) do.

Example (video Rental System)

Purpose

The purpose of the Software Requirements Specification document is to clearly define the system under development, namely the Video Rental System (VRS). The intended audience of this document includes the owner of the video store, the clerks of the video store, and the end users of the VRS.

*Other intended audience includes the development team such as the requirements team, requirements analyst, design team, and other members of the developing organization.

Scope

The owner of a local video store wanted to create a new business plan where everything about renting a video (except the picking up and returning of videos) was done online.

Therefore, the new VRS will allow the following functionality online:

- to search for videos,
 - to become members,
 - to rent videos,
 - to modify membership information,
 - and to pay overdue fees.
- The store personnel may use the VRS to process the rented or returned videos, to add or remove videos to/from his store's video inventory and to update video information.
- The VRS is intended to increase the owner's profit margin by increasing video sales with this unique business approach and by allowing him to reduce the staffing needed in his stores.

- **Definitions/acronyms/abbreviations**
 - ✓ Definitions of terms and abbreviations that are used in the SRS.
E.g.
 - ✓ User: The person operating and/or using the software system.
- **References**
 - ✓ A complete list of all documents referenced elsewhere in the SRS.
 - ✓ Specify the sources from which the references can be obtained.
- **Overview**
 - ✓ Brief description of rest of SRS.
 - ✓ How the SRS is organized

2.OVERALL DESCRIPTION

- **Product Perspective**
 - ✓ If the product is independent and totally self-contained, it should be stated here.
 - ✓ Describe the functions of each component of the larger system or project, and identify interfaces.
- **Product Functions**
 - ✓ Provide a summary of the functions that the software will perform.
- **User Characteristics**
 - ✓ Describe those general characteristics of the eventual users of the product that will affect the specific requirements.

- **General Constraints**
 - ✓ Provide a general description of any other items that will limit the **developer's options for designing the system.**
 - ✓ In this section, the constraints of the system are listed. They include **hardware**, **network**, **system software**, and **software constraints**. It also includes **user constraints**, **processing constraints**, **timing constraints**, and **control limits**.

E.g.

1. The software system will run under Windows.
2. All code shall be written in Java.

- **Assumptions and Dependencies**

- ✓ **List and description of each of the factors that affect the requirements stated in the SRS.**

Example

A Product Perspective

- *The VRS is a web-based system.*
- *The system interfaces with two other systems, the owner's email system, the video distributor's video system, and the browsers used by VRS customers.*
- *The system provides a secure environment for all financial transactions and for the storing and retrieving of confidential member information.*

Example

Product Functions

The VRS allows customers to search the video inventory provided by this video store. To rent videos through the VRS, one must register as a member using the VRS. Upon becoming a member and logging into the VRS, the VRS provides the functionality for renting videos, modifying membership information, and paying overdue fines.

The clerks of the video store use VRS to process the return of rented videos. The owner of the video store uses VRS to add new videos into the system, remove videos from the system, and modify video information.

The VRS sends emails to members concerning video rentals. One day before a rented video is due to be returned, VRS emails the member a reminder of the due date for the video(s). For any overdue videos, VRS emails the member every 3rd day with overdue notices. At the 60-day limit for outstanding videos, VRS debits the member's credit card with the appropriate charge and notifies the member of this charge.

Systems Requirements Specification

User Characteristics

The three main groups of VRS users are customers, members, and store personnel. A customer is anyone who is not a member. The customer can only search through the video inventory. The amount of product training needed for a customer is none since the level of technical expertise and educational background is unknown. The only skill needed by a customer is the ability to browse a website.

Member is someone who has registered with VRS. A member can rent videos and pay fees online. As with a customer, these activities require no product training since the level of technical expertise and educational background of a member is unknown. The only skill needed by a member is the ability to browse a website.

The store personnel are divided into two groups: the clerk-level personnel and owner-level personnel. Their educational level is unknown and both group needs little to no training.

Systems Requirements Specification

General Constraints

This system provides web access for all customer and member functions. The user interface will be intuitive enough so that no training is required by customers, members, or store personnel. All online financial transactions and the storage of confidential member information will be done in a secure environment. Persistent storage for membership, rental, and video inventory information will be maintained.

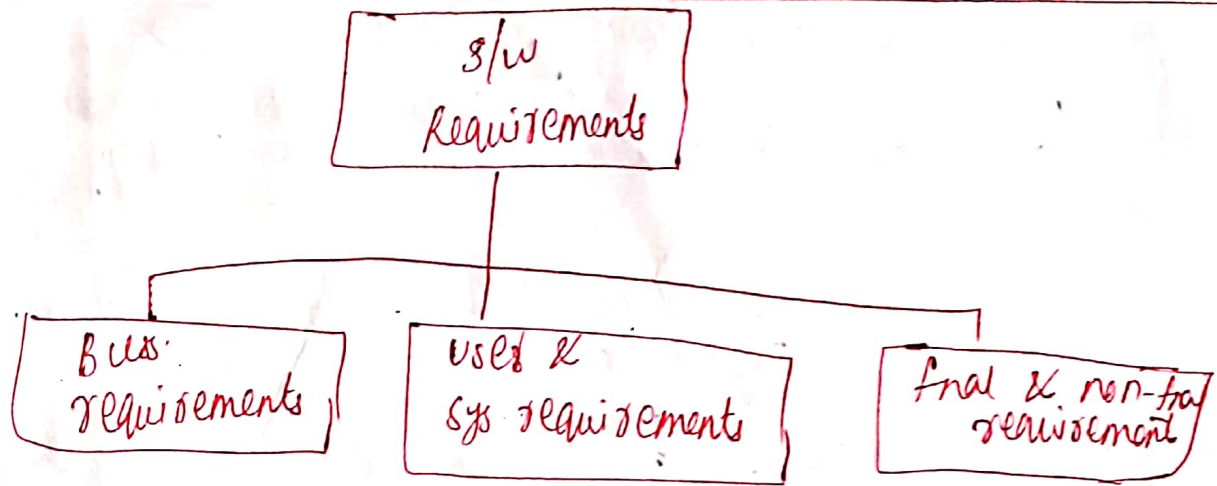


fig: Types of requirements

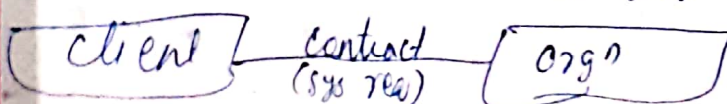
Bus. requirements:-

- profit goal & expected bus. benefits
- bus. needs
- prod. domain & prod. demand.

— Bus. anal.

User Requirements & sys. Requirements:-

- supplied by the customer, end-users, or other stakeholders
- Sys Req — functionalities that the sys. is expected to provide within certain constraints / environment.
- User Req $\begin{cases} \text{final} \\ \text{non-final} \end{cases}$
 - h/w & s/w
- sys. Req — technical functionalities written in a systematic manner to achieve user req.



IP.T.O.

③ Functional & Non-functional Req.:-

↳ to complete the product domain
↳ features which differentiate the sys. from other's products

functional — behavior / frs that the sys. must support.

Eg: services
tasks
behaviors.

non-functional — specify how a sys. must behave

Eg: Qualities
std's
constraints

Requirements Anal.:-

- * Analyzing stakeholder's needs,
 - constraints
 - assumptions
 - qualifications.

Adv: Refines & defines req. in a clear and unambiguous manner.

Types:

1. Structured analysis
2. Data-oriented
3. obj-oriented
4. prototyping

[Contd...]

Structured Analysis:- (process modeling / data flow modeling)

1. DFD - sys. behavior
2. Data Dictionary - Composite data structures defined in DFD.

→ symbols :

- + → composition
- * → repetition
- | → selection
- [] → combination of repeated data

Eg: restaurant bill

- restaurant_bill = dish_price + sales_tax + service_charge + seat_charge + grand_tot.
- dish_price = [dish_name + quantity + price] *
- seat_charge = [reserved_table | common_table]

- DFD + DD's
3. structured Anal. method - techniques and graphical tools to represent the scenario of the working sys. in an orgn

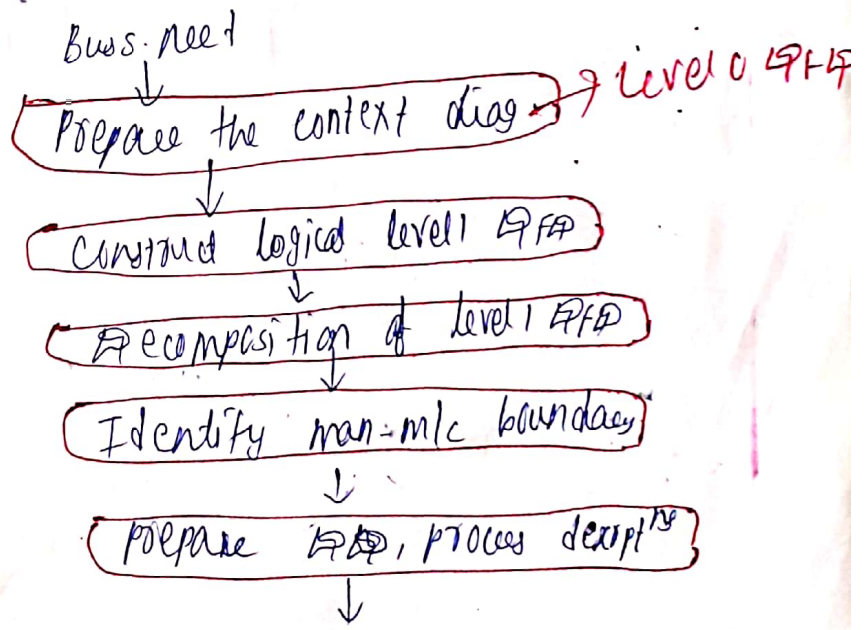


Fig: structured anal. approach

Requirements specification

(P-T-C)

② Data-oriented Anal.:-

1. E-R modeling (ERM)

Entities attributes R-s's

Aggregatⁿ → part-of / part-whole R-s's b/w entities

Specialization → is-a R-skip.

Generalization →

2. Data-oriented Anal. method.

- Identification of entity & R-s's
- Construct the basic E-R diag
- Add key attributes
- Add non-key
- Apply hierarchical relation
- Perform normalization
- Add integrity rules

③ Obj-oriented Anal.:-

OOA - Engg. Req, modeling the app^{re}

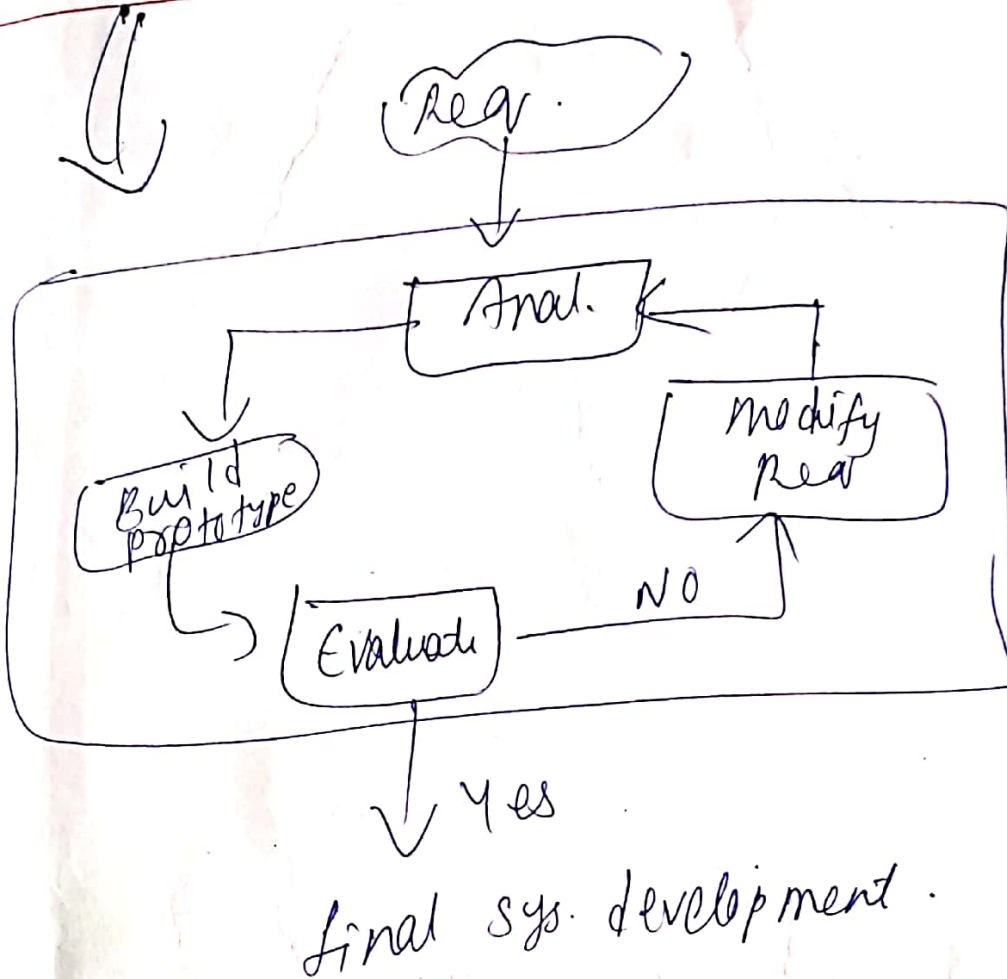
OOD - translates OOA into pgm^{ing}. Constructs to produce practical

- class diag
- obj modeling
- dynamic modeling : seq. diag
- functional modeling : st-chart diag

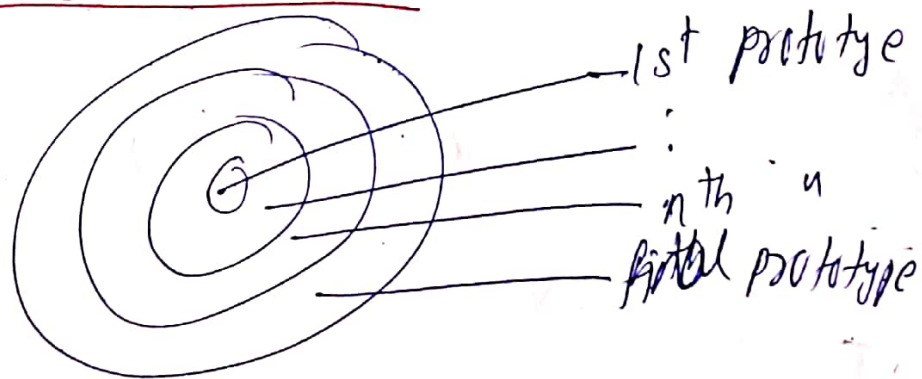
Contd--

④ Prototyping: Throwaway prototyping
Evolutionary

① Throwaway Prototyping:



② Evolutionary prototyping: Iterative



→ use RAA

SRS specific methods.

- ① A. Tree
- ② A. Table

①

③ PAL - specifies procedures of functional requirements in slw.

- Derived from Ada
- Pseudo code.

Eg: SPS

Procedure SPS is

BEGIN

LOOP

read student_result(credits, sgpa)

IF credits is greater than 52

THEN calculate sgpa

ELSE

msg incomplete credit

ENDIF

IF sgpa is greater than 4.00

THEN promote()

ELSE readmit()

ENDIF

END LOOP

④ Regular Expr - describes syntactical structure of a set of strings.

P.T.O

⑤ Graphical methods — PFD, ER, FDM (represents behavior of a sys.)

⑥ Mathematical methods — used to model complex sys's in mathematical technol^y.

- algebraic specificaⁿ
- axiomatic^y
- theorems.

Specificaⁿ = (Sort, Operⁿ, Axioms);

set of values
that describes
data type or class

name of
the operⁿ,
its parameters
and return type

rules that must
hold true in any
legal implementaⁿ
of sort.

Requirements validaⁿ:- To avoid ~~an~~ inconsistency,
incorrect fact, ambiguity, omission of unnecessary
requirements

→ ① Requirements Review:-

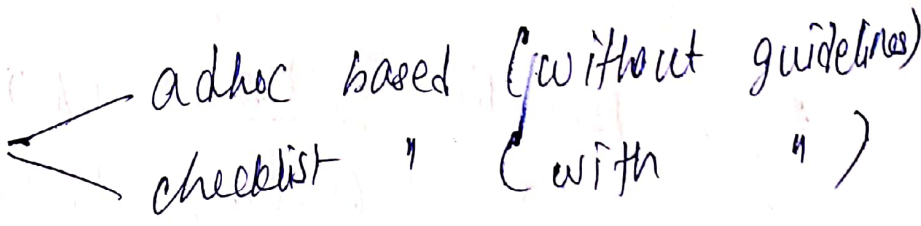
- ↳ done by group of people
- ↳ involves readers from both sides
(clients & developer)

- Review plan
- " team
- Time & place

② Requirements Inspection:- detects defects at early stages. (up to 90%)

- moderator keeps doc's ready
- inspectors inspect
- organiser holds meeting

③ T-c's Generation:- Black box test cases are created.

④ Reading: 

- Ambiguity
- understandability
- completeness
- Redundancy
- Testability
- modifiability → Traceability

→ ⑧ Prototyping:- understand, analyse, and validate req's.

Prototyping validation — identifies errors like missing, incomplete and incorrect req's from req. doc.

Req's mgmt:- Process of systematically collecting, organising, documenting, prioritizing, and negotiating on the req's for a projt.

Chapter 3

Planning and Managing the Project

Contents

- 3.1 Tracking Progress**
- 3.2 Project Personnel**
- 3.3 Effort Estimation**
- 3.4 Risk Management**
- 3.5 The Project Plan**
- 3.6 Process Models and Project Management**
- 3.7 Information System Example**
- 3.8 Real Time Example**
- 3.9 What this Chapter Means for You**

Chapter 3 Objectives

- **Tracking project progress**
- **Project personnel and organization**
- **Effort and schedule estimation**
- **Risk management**
- **Using process modeling with project planning**

3.1 Tracking Progress

- Do we **understand customer's needs**?
- Can we design a system to **solve customer's problems** or **satisfy customer's needs**?
- How long will it take to develop the system?
- How much will it cost to develop the system?

3.1 Tracking Progress

Project Schedule

- **Describes** the software-development cycle for a particular project by
 - 1) **enumerating** the **phases or stages** of the project.
 - 2) **breaking** each **phase** into **discrete tasks or activities** to be completed.
- **Portrays** the **interactions** among the **activities** and **estimates** the **times** that each task or activity will take.

3.1 Tracking Progress

Project Schedule: Approach

- **Understanding customer's needs** by listing all project deliverables
 - 1) Documents
 - 2) Demonstrations of function
 - 3) Demonstrations of subsystems
 - 4) Demonstrations of accuracy
 - 5) Demonstrations of reliability, performance or security
- **Determining milestones** and **activities** to produce the deliverables.

3.1 Tracking Progress

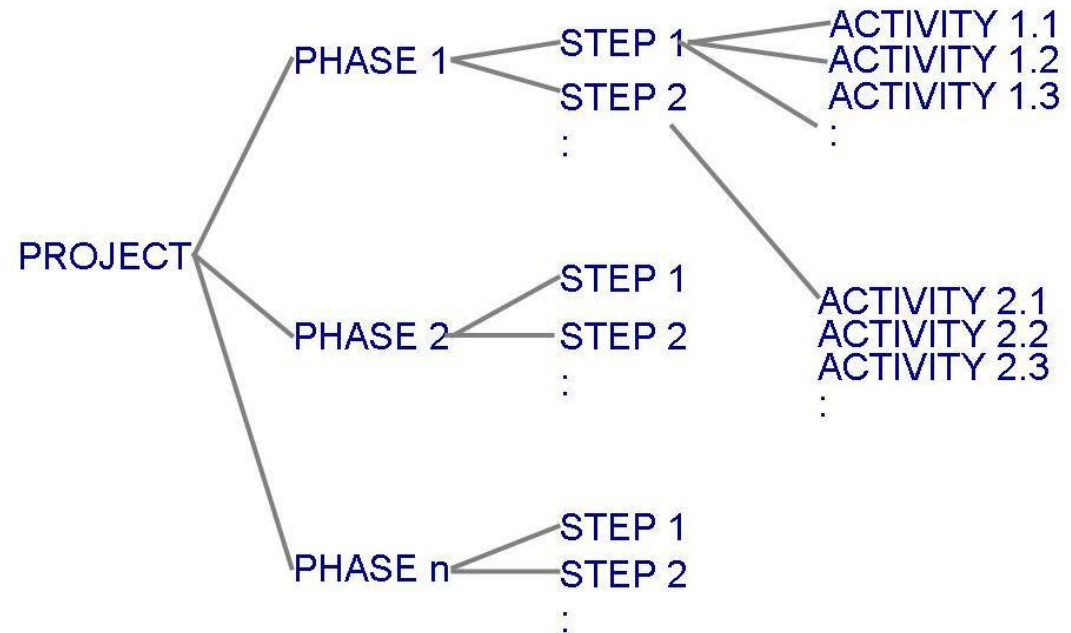
Milestones and activities

- **Activity**: takes place over a period of time.
- **Milestone**: completion of an activity -- a particular point in time.
- **Precursor**: **event or set of events** that must occur in order for an **activity to start**.
- **Duration**: length of time needed to complete an activity.
- **Due date**: date by which an activity must be completed.

3.1 Tracking Progress

Project Schedule

- **Project** development can be separated into a succession of **phases** which are composed of **steps**, which are composed of **activities**.



3.1 Tracking Progress

Project Schedule

- **Table 3.1 shows the phases, steps and activities to build a house**
 - **landscaping phase**
 - **building the house phase**
- **Table 3.2 lists milestones for building the house phase**

3.1 Tracking Progress

Phases, Steps, and Activities in Building a House

Phase 1: Landscaping the lot			Phase 2: Building the house		
<i>Step 1.1: Clearing and grubbing</i>			<i>Step 2.1: Prepare the site</i>		
Activity 1.1.1: Remove trees			Activity 2.1.1: Survey the land		
Activity 1.1.2: Remove stumps			Activity 2.1.2: Request permits		
	<i>Step 1.2: Seeding the turf</i>		Activity 2.1.3: Excavate for the foundation		
Activity 1.2.1: Aerate the soil			Activity 2.1.4: Buy materials		
Activity 1.2.2: Disperse the seeds				<i>Step 2.2: Building the exterior</i>	
Activity 1.2.3: Water and weed			Activity 2.2.1: Lay the foundation		
		<i>Step 1.3: Planting shrubs and trees</i>	Activity 2.2.2: Build the outside walls		
Activity 1.3.1: Obtain shrubs and trees			Activity 2.2.3: Install exterior plumbing		
Activity 1.3.2: Dig holes			Activity 2.2.4: Exterior electrical work		
Activity 1.3.3: Plant shrubs and trees			Activity 2.2.5: Exterior siding		
Activity 1.3.4: Anchor the trees and mulch around them			Activity 2.2.6: Paint the exterior		
			Activity 2.2.7: Install doors and fixtures		
			Activity 2.2.8: Install roof		
				<i>Step 2.3: Finishing the interior</i>	
			Activity 2.3.1: Install the interior plumbing		
			Activity 2.3.2: Install interior electrical work		
			Activity 2.3.3: Install wallboard		
			Activity 2.3.4: Paint the interior		
			Activity 2.3.5: Install floor covering		
			Activity 2.3.6: Install doors and fixtures		

3.1 Tracking Progress

Milestones in Building a House

1.1. Survey complete
1.2. Permits issued
1.3. Excavation complete
1.4. Materials on hand
2.1. Foundation laid
2.2. Outside walls complete
2.3. Exterior plumbing complete
2.4. Exterior electrical work complete
2.5. Exterior siding complete
2.6. Exterior painting complete
2.7. Doors and fixtures mounted
2.8. Roof complete
3.1. Interior plumbing complete
3.2. Interior electrical work complete
3.3. Wallboard in place
3.4. Interior painting complete
3.5. Floor covering laid
3.6. Doors and fixtures mounted

3.1 Tracking Progress

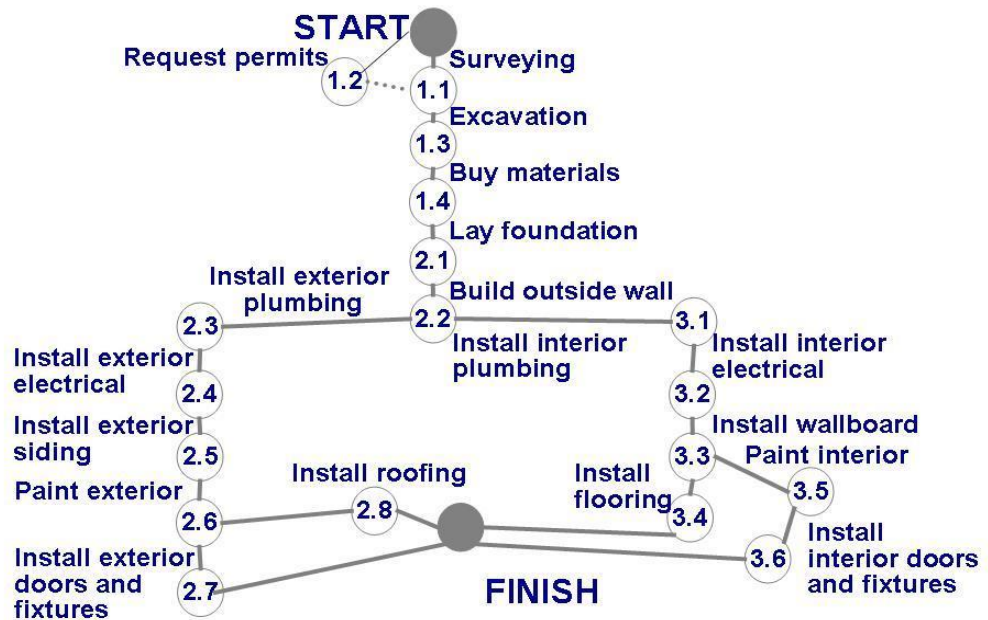
Work Breakdown and Activity Graphs

- **Work breakdown** structure depicts the project as a set of discrete pieces of work.
- **Activity graphs** depict the dependencies among activities.
 - **Nodes**: project milestones.
 - **Lines**: activities involved.

3.1 Tracking Progress

Work Breakdown and Activity Graphs

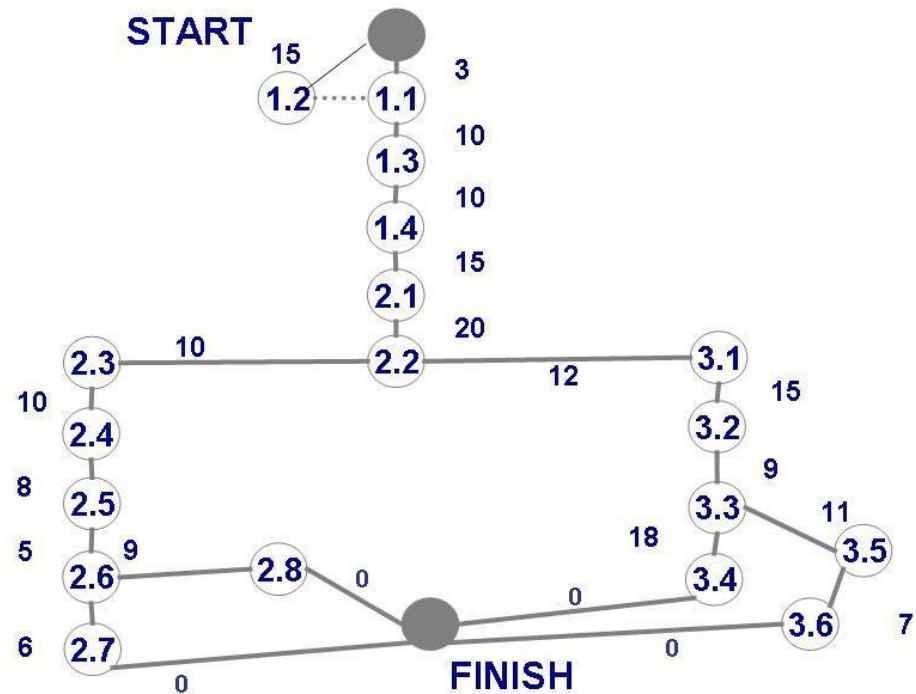
- Activity graph for building a house



3.1 Tracking Progress

Estimating Completion

- Adding **estimated time** in activity graph of each activity to be completed tells us more about the project's schedule.



3.1 Tracking Progress

Estimating Completion for Building a House

Activity	Time estimate (in days)
<i>Step 1: Prepare the site</i>	
Activity 1.1: Survey the land	3
Activity 1.2: Request permits	15
Activity 1.3: Excavate for the foundation	10
Activity 1.4: Buy materials	10
<i>Step 2: Building the exterior</i>	
Activity 2.1: Lay the foundation	15
Activity 2.2: Build the outside walls	20
Activity 2.3: Install exterior plumbing	10
Activity 2.4: Exterior electrical work	10
Activity 2.5: Exterior siding	8
Activity 2.6: Paint the exterior	5
Activity 2.7: Install doors and fixtures	6
Activity 2.8: Install roof	9
<i>Step 3: Finishing the interior</i>	
Activity 3.1: Install the interior plumbing	12
Activity 3.2: Install interior electrical work	15
Activity 3.3: Install wallboard	9
Activity 3.4: Paint the interior	18
Activity 3.5: Install floor covering	11
Activity 3.6: Install doors and fixtures	7

3.1 Tracking Progress

Critical Path Method (CPM)

- Minimum amount of **time** it will take to **complete a project**
 - **Reveals** those activities that are most critical to completing the project on time.
- **Real time (actual time)**: estimated amount of time required for the activity to be completed.
- **Available time**: amount of time available in the schedule for the activity's completion
- **Slack time**: the difference between the available time and the real time for that activity.

3.1 Tracking Progress

Critical Path Method (CPM)

- Critical path: the slack at every node is zero
 - can be more than one in a project schedule
- Slack time = available time – real time
 - = latest start time – earliest start time

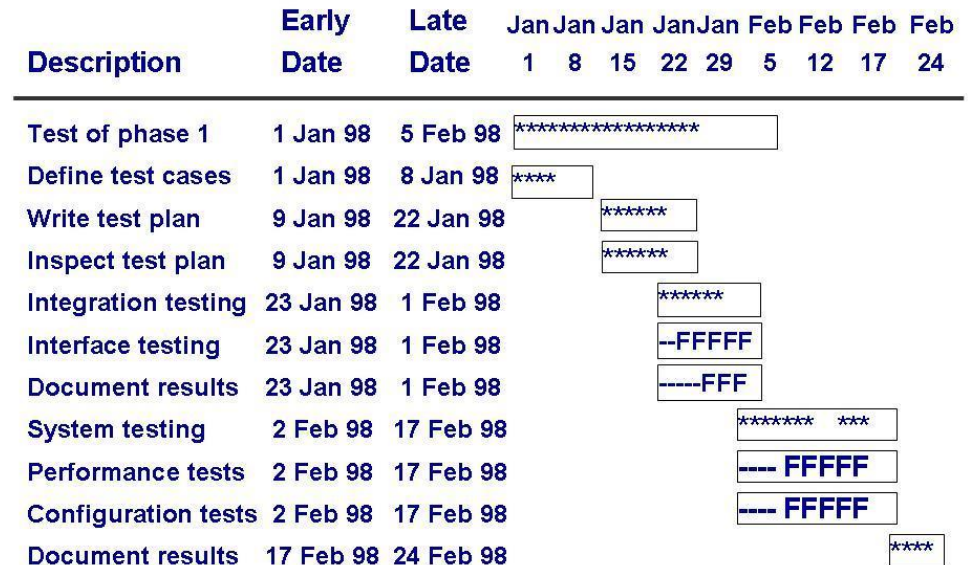
Slack Time for Activities of Building a House

Activity	Earliest start time(RT)	Latest start time(AT)	Slack
1.1	1	13	12
1.2	1	1	0
1.3	16	16	0
1.4	26	26	0
2.1	36	36	0
2.2	51	51	0
2.3	71	83	12
2.4	81	93	12
2.5	91	103	12
2.6	99	111	12
2.7	104	119	15
2.8	104	116	12
3.1	71	71	0
3.2	83	83	0
3.3	98	98	0
3.4	107	107	0
3.5	107	107	0
3.6	118	118	0
Finish	124	124	0

3.1 Tracking Progress

CPM Bar Chart

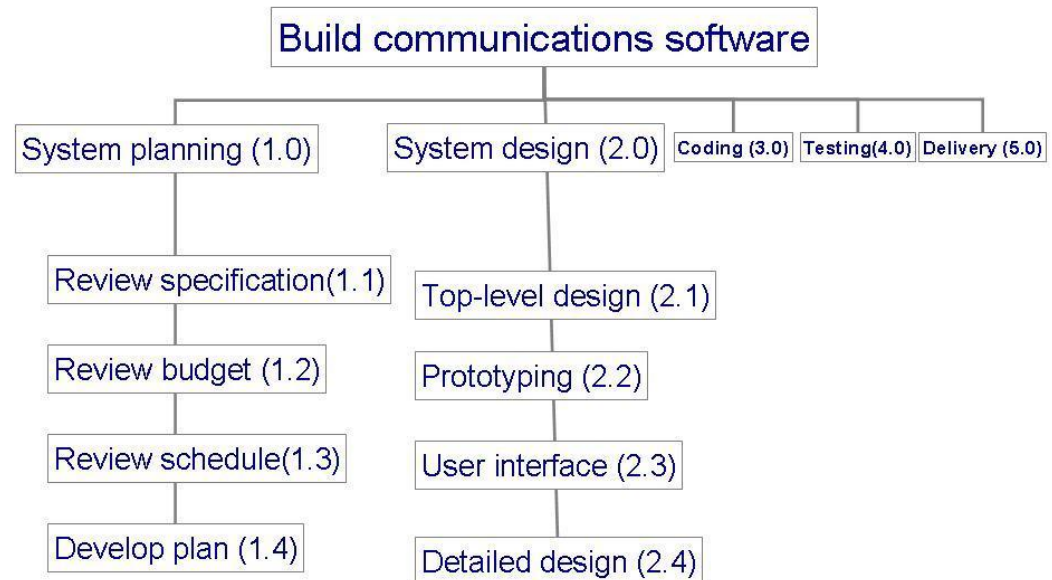
- Including information about the early and late start dates
- Asterisks indicate the critical path



3.1 Tracking Progress

Tools to Track Progress

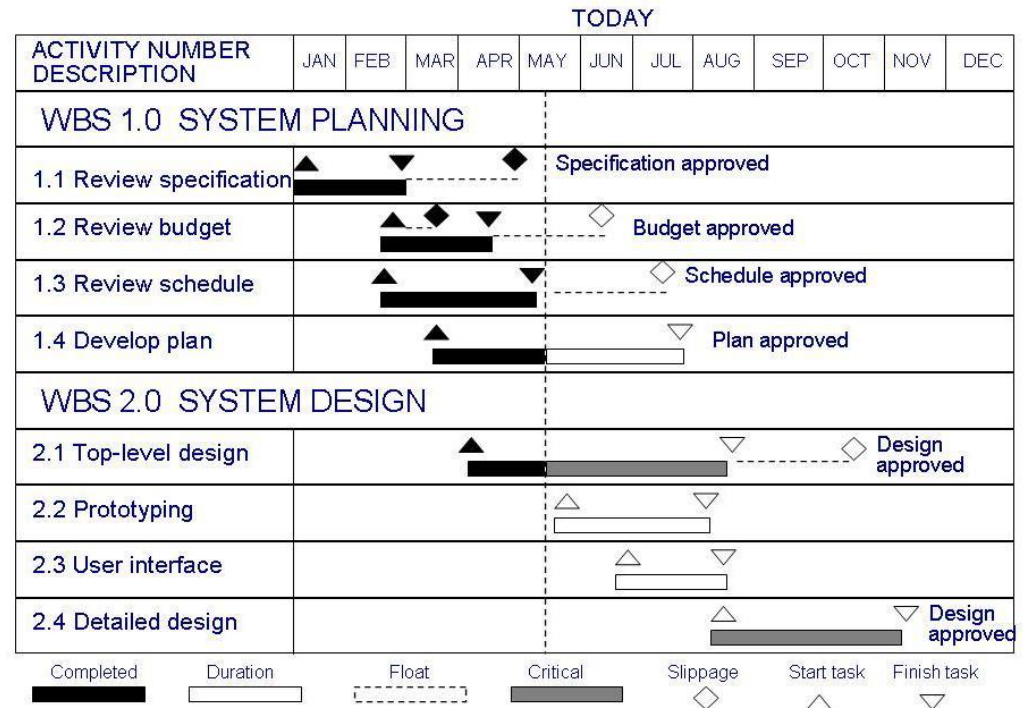
- Example:
to track
progress of
building a
communication
software



3.1 Tracking Progress

Tools to Track Progress: Gantt Chart

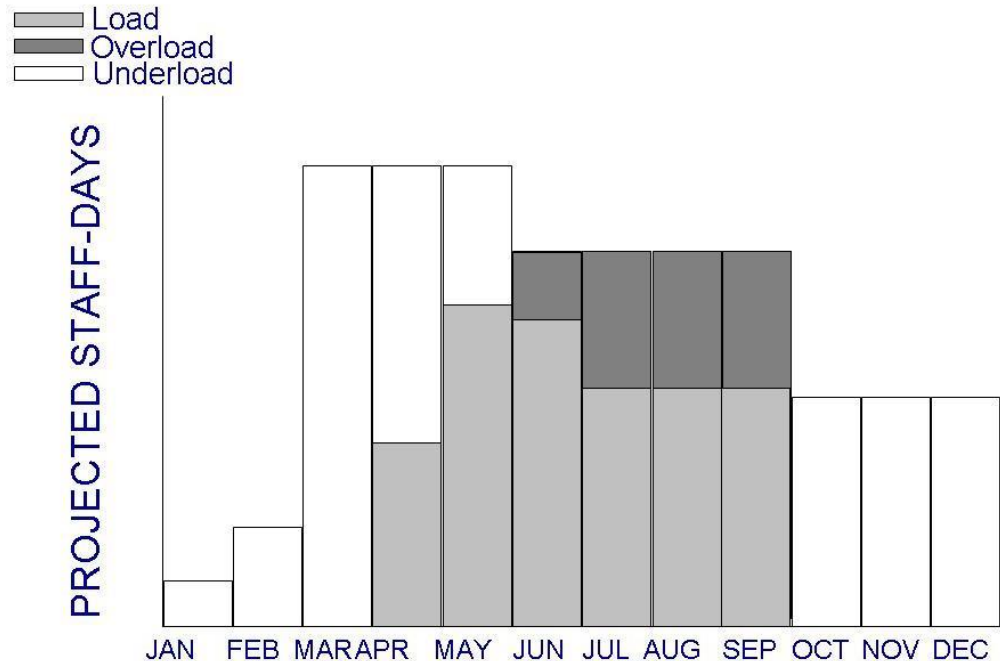
- Activities shown in parallel
 - helps understand which activities can be performed concurrently



3.1 Tracking Progress

Tools to Track Progress: Resource Histogram

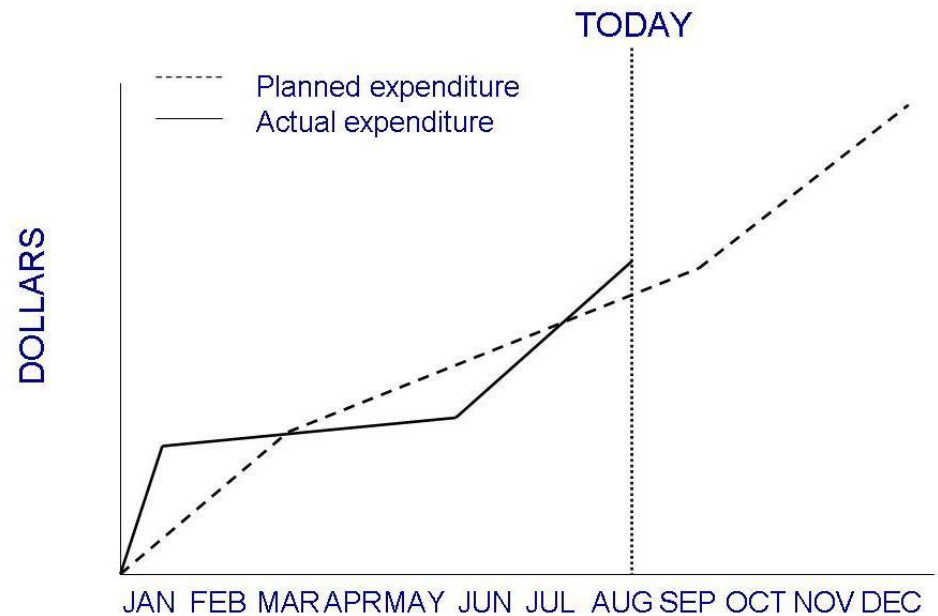
- Shows people assigned to the project and those needed for each stage of development



3.1 Tracking Progress

Tools to Track Progress: Expenditures Tracking

- An example of how expenditures can be monitored



3.2 Project Personnel

- **Key activities requiring personnel**
 - requirements analysis
 - system design
 - program design
 - program implementation
 - testing
 - training
 - maintenance
 - quality assurance
- **There is great advantage in assigning different responsibilities to different people.**

3.2 Project Personnel

Choosing Personnel

- **Ability to perform work.**
- **Interest** in work.
- **Experience** with
 - similar applications
 - similar tools, languages, or techniques
 - similar development environments
- **Training.**
- Ability to **communicate** with others.
- Ability to **share responsibility.**
- **Management skills.**

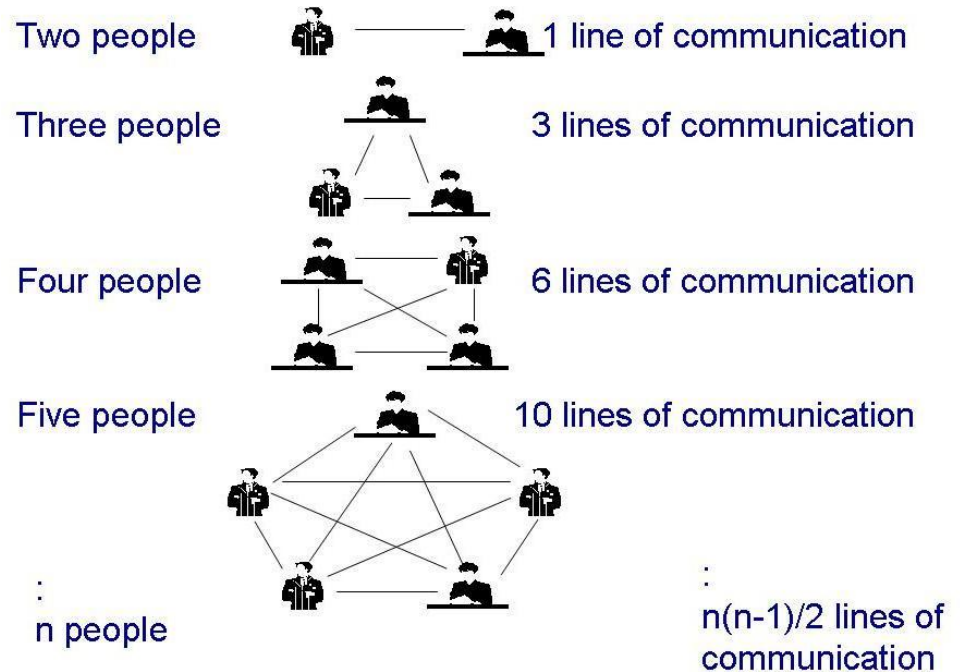
3.2 Project Personnel

Communication

- A project's **progress** is affected by
 - **degree of communication.**
 - **ability of individuals to communicate their ideas.**
- S/w **failures** can result from **breakdown in communication and understanding.**

3.2 Project Personnel Communication

- **Line of communication** can grow quickly
- If there is n worker in project, then there are $n(n-1)/2$ pairs of communication



3.2 Project Personnel

Sidebar 3.1 Make Meeting Enhance Project Progress

- **Common complains about meeting**
 - the purpose is unclear
 - the attendees are unprepared
 - essential people are late or absent
 - the conversation veers away from its purpose
 - participants do not discuss, instead argue
 - decisions are never enacted afterward
- **Ways to ensure a productive meeting**
 - clearly decide who should be in the meeting
 - develop an agenda
 - have someone who tracks the discussion
 - have someone who ensures follow-up actions

3.2 Project Personnel

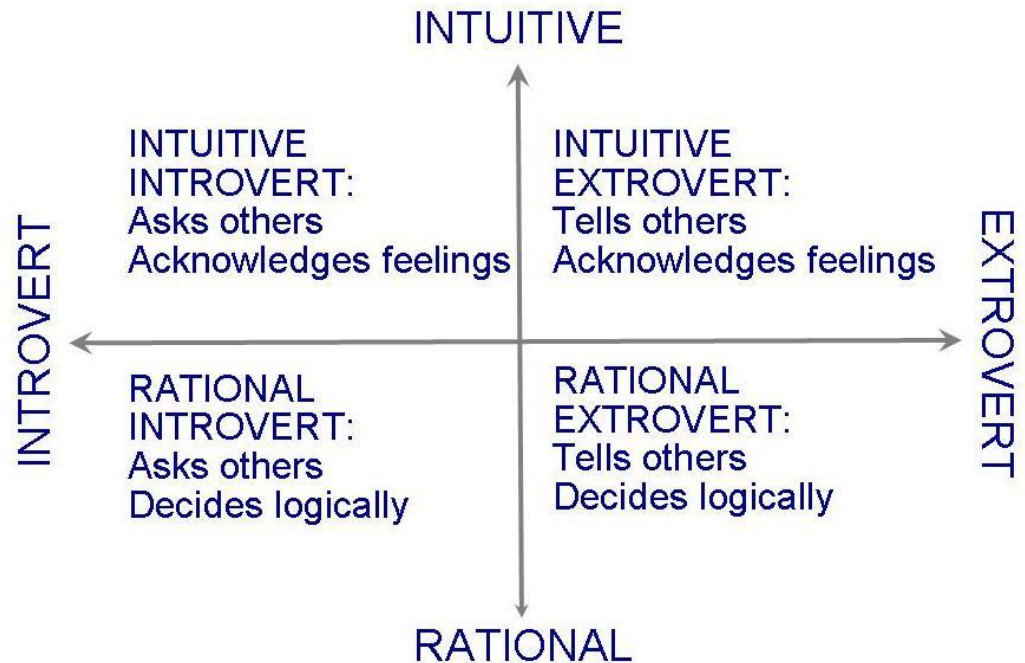
Work Styles

- 1) **Extroverts:** tell their thoughts.
- 2) **Introverts:** ask for suggestions.
- 3) **Intuitives:** base decisions on feelings.
- 4) **Rationals:** base decisions on facts, options.

3.2 Project Personnel

Work Styles

- Horizontal axis: communication styles
- Vertical axis: decision styles



3.2 Project Personnel

Work Styles

- **Work styles** determine **communication styles**
- Understanding workstyles
 - help to be flexible
 - give information based on other's priorities
- Impacts interaction among customers, developers and users

3.2 Project Personnel

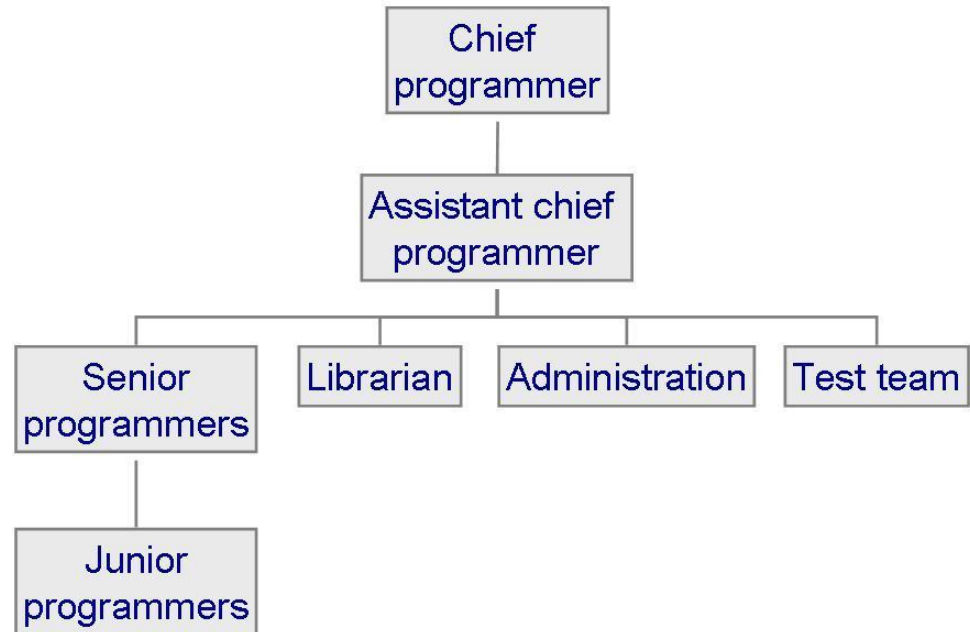
Project Organization

- Depends on
 - 1) **backgrounds** and **work styles** of team members.
 - 2) number of people on team.
 - 3) **management styles** of customers and developers.
- Examples:
 - **Chief programmer team:** one person totally responsible for a system's design and development
 - **Egoless approach:** *hold everyone equally responsible*

3.2 Project Personnel

Project Organization: Chief Programmer Team

- Each team member must communicate often with chief, but not necessarily with other team members



3.2 Project Personnel

Project Organization

- **Characteristics of projects** and the **suggested organizational structure** to address them

Highly structured	Loosely structured
High certainty	Uncertainty
Repetition	New techniques or technology
Large projects	Small projects

3.2 Project Personnel

Sidebar 3.2 Structure vs. Creativity

- Experiment by Sally Phillip examining two groups building a hotel
 - structured team: clearly defined responsibilities.
 - unstructured team: no directions.
- The results are always the same
 - Structured teams *finish* a functional Days Inn.
 - Unstructured teams build a **creative, multistoried** Taj Mahal and never complete.
- Good project management means finding a balance between structure and creativity.

3.3 Effort Estimation

- Estimating project costs is one of the crucial aspects of project planning and management.
- Estimating cost has to be done as early as possible during the project life cycle.
- Type of costs
 - 1) facilities: hardware, space, furniture, telephone, etc.
 - 2) software tools: for designing software.
 - 3) staff (effort): the biggest component of cost.

3.3 Effort Estimation

Estimation Should be Done Repeatedly

- Uncertainty early in the project can affect the accuracy of cost and size estimations



3.3 Effort Estimation

Sidebar 3.3 Causes of Inaccurate Estimates

- Key causes(6)
 - 1) Frequent request for change by users.
 - 2) Overlooked(missed/unnoticed) tasks.
 - 3) User's lack of understanding of the requirements.
 - 4) Insufficient analysis when developing estimates.
 - 5) Lack of coordination of system development, technical services, operations, data administration, and other functions during development.
 - 6) Lack of an adequate method or guidelines for estimating.

3.3 Effort Estimation

Sidebar 3.3 Causes of Inaccurate Estimates

- Key influences(10)
 - Complexity of the proposed application system.
 - Required integration with existing system.
 - Complexity of the program in the system.
 - Size of the system expressed as number of functions or programs.
 - Capabilities of the project team members.

–

Cont...

- Cont...
- Project team's experience with the application, the programming language, and hardware.
- Capabilities of the project team members.
- Database management system.
- Number of project team member.
- Extent of programming and documentation standards.

3.3 Effort Estimation

Type of Estimation Methods

1) Expert judgment

- **Top-down or bottom-up**
- **Analogy**: pessimistic (x), optimistic (y), most likely (z);
estimate as $(x + 4y + z)/6$
- **Delphi technique**: based on the average of “secret” expert judgments

2) Algorithmic methods: $E = (a + bS^c) m(X)$

- **Walston and Felix model**: $E = 5.25 S^{0.91}$
- **Bailey and Basili model**: $E = 5.5 + 0.73 S^{1.16}$

3.3 Effort Estimation

Expert Judgement: Wolverton Model

- Two factors that affect difficulty
 - whether problem is **old (O)** or **new (N)**
 - whether it is **easy (E)** or **moderate (M)** or **hard(H)**

	<i>Difficulty</i>					
<i>Type of software</i>	<i>OE</i>	<i>OM</i>	<i>OH</i>	<i>NE</i>	<i>NM</i>	<i>NH</i>
Control	21	27	30	33	40	49
Input/output	17	24	27	28	35	43
Pre/post processor	16	23	26	28	34	42
Algorithm	15	20	22	25	30	35
Data management	24	31	35	37	46	57
Time-critical	75	75	75	75	75	75

3.3 Effort Estimation

Algorithmic Method: Watson and Felix Model

- A productivity index is included in the equation.
- There are 29 factors that can affect productivity
 - 1 if increase the productivity
 - 0 if decrease the productivity

3.3 Effort Estimation

Watson and Felix Model Productivity Factors

1. Customer interface complexity	16. Use of design and code inspections
2. User participation in requirements definition	17. Use of top-down development
3. Customer-originated program design changes	18. Use of a chief programmer team
4. Customer experience with the application area	19. Overall complexity of code
5. Overall personnel experience	20. Complexity of application processing
6. Percentage of development programmers who participated in the design of functional specifications	21. Complexity of program flow
7. Previous experience with the operational computer	22. Overall constraints on program's design
8. Previous experience with the programming language	23. Design constraints on the program's main storage
9. Previous experience with applications of similar size and complexity	24. Design constraints on the program's timing
10. Ratio of average staff size to project duration (people per month)	25. Code for real-time or interactive operation or for execution under severe time constraints
11. Hardware under concurrent development	26. Percentage of code for delivery
12. Access to development computer open under special request	27. Code classified as nonmathematical application and input/output formatting programs
13. Access to development computer closed	28. Number of classes of items in the database per 1000 lines of code
14. Classified security environment for computer and at least 25% of programs and data	29. Number of pages of delivered documentation per 1000 lines of code
15. Use of structured programming	

3.3 Effort Estimation

Algorithmic Method: Bailey-Basili technique

- Minimize standard error estimate to produce an equation such as
$$E = 5.5 + 0.73S^{1.16}$$
- Adjust initial estimate based on the difference ratio
 - If R is the ratio between the **actual effort**, E , and the **predicted effort**, E' , then the effort adjustment is defined as
 - $ER_{adj} = R - 1$ if $R > 1$
 $= 1 - 1/R$ if $R < 1$
- Adjust the initial effort estimate E_{adj}
 - $E_{adj} = (1 + ER_{adj})E$ if $R > 1$
 $= E/(1 + ER_{adj})$ if $R < 1$

3.3 Effort Estimation

Algorithmic Method: Bailey-Basily Modifier

<i>Total methodology (METH)</i>	<i>Cumulative complexity (CPLX)</i>	<i>Cumulative experience (EXP)</i>
Tree charts	Customer interface complexity	Programmer qualifications
Top-down design	Application complexity	Programmer machine experience
Formal documentation	Program flow complexity	Programmer language experience
Chief programmer teams	Internal communication complexity	Programmer application experience
Formal training	Database complexity	Team experience
Formal test plans	External communication complexity	
Design formalisms	Customer-initiated program design changes	
Code reading		
Unit development folders		

3.3 Effort Estimation

COCOMO model

- Introduced by Boehm
- COCOMO II
 - updated version
 - include models of reuse
- The basic models
 - **$E = bS^c m(X)$**
 - where
 - **bS^c** is the initial size-based estimate
 - **$m(X)$** is the vector of cost driver information

3.3 Effort Estimation

COCOMO II: Stages of Development

- **Application composition**
 - prototyping to resolve high-risk user interface issues.
 - size estimates in object points.
- **Early design**
 - to explore alternative architectures and concepts.
 - size estimates in function points.
- **Post architecture**
 - development has begun.
 - size estimates in lines of code.

Three Stages of COCOMO II

Model Aspect	Stage 1: Application Composition	Stage 2: Early Design	Stage 3: Post-architecture
Size	Application points	Function points (FP) and language	FP and language or source lines of code (SLOC)
Reuse	Implicit in model	Equivalent SLOC as function of other variables	Equivalent SLOC as function of other variables
Requirements change	Implicit in model	% change expressed as a cost factor	% change expressed as a cost factor
Maintenance	Application Point Annual Change Traffic	Function of ACT, software understanding, unfamiliarity	Function of ACT, software understanding, unfamiliarity
Scale (c) in nominal effort equation	1.0	0.91 to 1.23, depending on precedentedness, conformity, early architecture, risk resolution, team cohesion, and SEI process maturity	0.91 to 1.23, depending on precedentedness, conformity, early architecture, risk resolution, team cohesion, and SEI process maturity
Product cost drivers	None	Complexity, required reusability	Reliability, database size, documentation needs, required reuse, and product complexity
Platform cost drivers	None	Platform difficulty	Execution time constraints, main storage constraints, and virtual machine volatility
Personnel cost drivers programmer experience, experience, and personnel continuity	None	Personnel capability and experience	Analyst capability, applications experience, programmer capability, language and tool
Project cost drivers	None environment	Required development schedule, development multisite development	Use of software tools, required development schedule, and

3.3 Effort Estimation

COCOMO II: Estimate Application Points

- To compute application points, first we need to **count the number of screens, reports and programming language** used to determine the complexity level.

For Screens				For Reports			
	Number and source of data tables				Number and source of data tables		
Number of views contained	Total < 4 (<2 server, <3 client)	Total < 8 (2-3 server, 3-5 client)	Total 8+ (>3 server, >5 client)	Number of sections contained	Total < 4 (<2 server, <3 client)	Total < 8 (2-3 server, 3-5 client)	Total 8+ (>3 server, >5 client)
<3	simple	simple	medium	0 or 1	simple	simple	medium
3 - 7	simple	medium	difficult	2 or 3	simple	medium	difficult
8 +	medium	difficult	difficult	4 +	medium	difficult	difficult

3.3 Effort Estimation

COCOMO II: Estimate Application Point

- Determine the relative effort required to implement a **report** or **screen** **simple**, **medium** or **difficult**.
- Calculate the **productivity factor** based on developer experience and capability.
- Determine the **adjustment factors** expressed as multipliers based on rating of the project.

3.3 Effort Estimation

Complexity Weights for Application Points

<i>Object type</i>	<i>Simple</i>	<i>Medium</i>	<i>Difficult</i>
Screen	1	2	3
Report	2	5	8
3GL component	-	-	10

3.3 Effort Estimation

Productivity Estimate Calculation

Developers' experience and capability	Very low	Low	Nominal	High	Very high
CASE maturity and capability	Very low	Low	Nominal	High	Very high
<i>Productivity factor</i>	4	7	13	25	50

3.3 Effort Estimation

Tool Use Categories

<i>Category</i>	<i>Meaning</i>
Very low	Edit, code, debug
Low	Simple front-end, back-end CASE, little integration
Nominal	Basic life-cycle tools, moderately integrated
High	Strong, mature life-cycle tools, moderately integrated
Very high	Strong, mature, proactive life-cycle tools, well-integrated with processes, methods, reuse

3.3 Effort Estimation

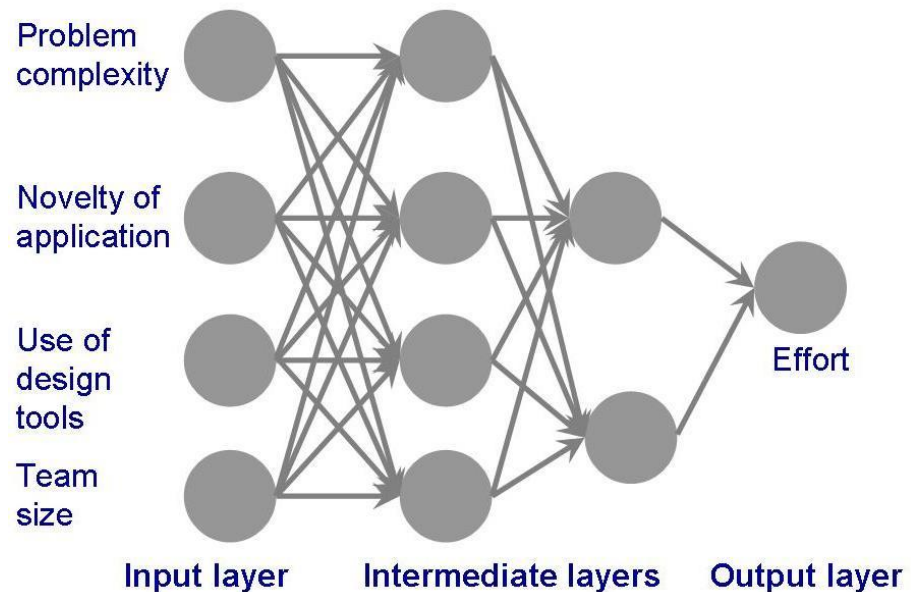
Machine Learning Techniques

- Example: **case-based reasoning (CBR)**
 - user identifies new problem as a case.
 - system retrieves similar cases from repository.
 - system reuses knowledge from previous cases.
 - system suggests solution for new case.
- Example: **neural network**
 - cause-effect network “trained” with data from past history

3.3 Effort Estimation

Machine learning techniques: Neural Network

- **Neural network** used by Shepperd to produce effort estimation



3.3 Effort Estimation

Machine Learning Techniques: CBR

- **Involves four steps**
 - 1) the user identifies a new problem as a case.
 - 2) the system retrieves similar case from a repository of historical information.
 - 3) the system reuses knowledge from previous case.
 - 4) the system suggests a solution for the new case.
- **Two big hurdles** in creating successful CBR system
 - 1) characterizing cases.
 - 2) determining similarity.

3.3 Effort Estimation

Finding the Model for Your Situation

- **Mean magnitude of relative error (MMRE)**
 - absolute value of mean of $[(\text{actual} - \text{estimate})/\text{actual}]$
 - goal: should be .25 or less
- **Pred($x/100$):** percentage of projects for which estimate is within $x\%$ of the actual
 - goal: should be .75 or greater for $x = .25$
 - 75% project estimates are within 25% of actual

3.3 Effort Estimation

Evaluating Models

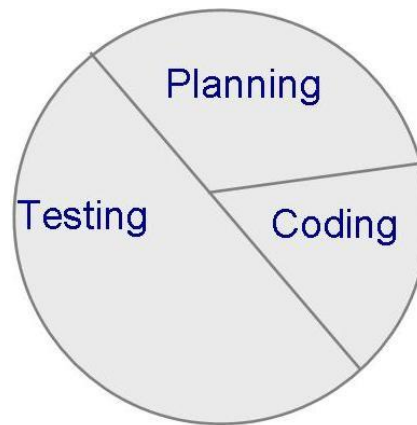
- No model appears to have captured the essential characteristics and their relationships for all types of development

<i>Model</i>	<i>PRED(0.25)</i>	<i>MMRE</i>
Walston-Felix	0.30	0.48
Basic COCOMO	0.27	0.60
Intermediate COCOMO	0.63	0.22
Intermediate COCOMO (variation)	0.76	0.19
Bailey-Basili	0.78	0.18
Pfleeger	0.50	0.29
SLIM	0.06-0.24	0.78-1.04
Jensen	0.06-0.33	0.70-1.01
COPMO	0.38-0.63	0.23-5.7
General COPMO	0.78	0.25

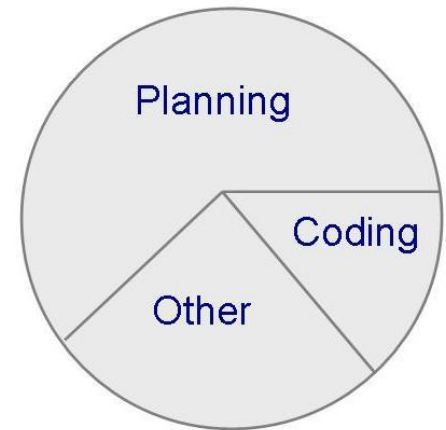
3.3 Effort Estimation

Evaluating Models

- It is important to understand which types of effort are needed during development even when we have reasonably accurate estimate
 - Categorize and save the results
- Two different reports of effort distribution from different researchers



Brooks



Yourdon

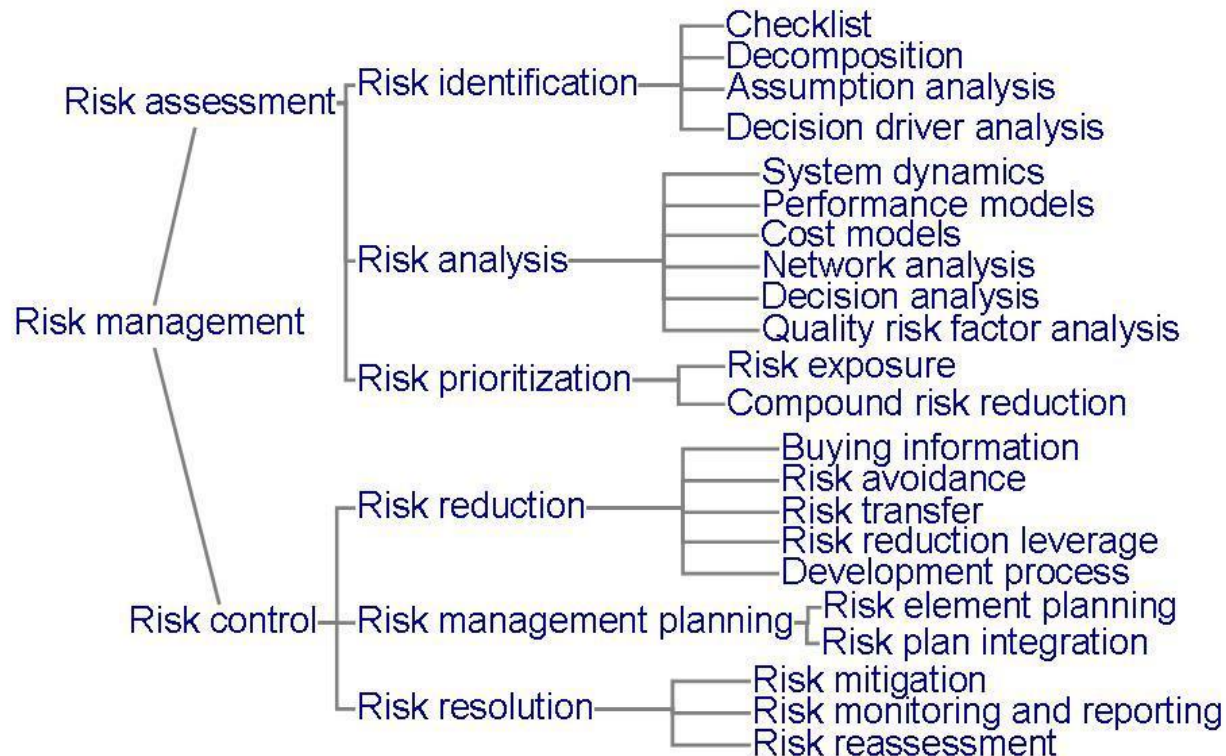
3.4 Risk Management

What is a Risk?

- Risk is an unwanted event that has negative consequences
- Distinguish risks from other project events
 - ***Risk impact***: the loss associated with the event
 - ***Risk probability***: the likelihood that the event will occur
- Quantify the effect of risks
 - ***Risk exposure = (risk probability) x (risk impact)***
- **Risk sources**: generic and project-specific

3.4 Risk Management

Risk Management Activities



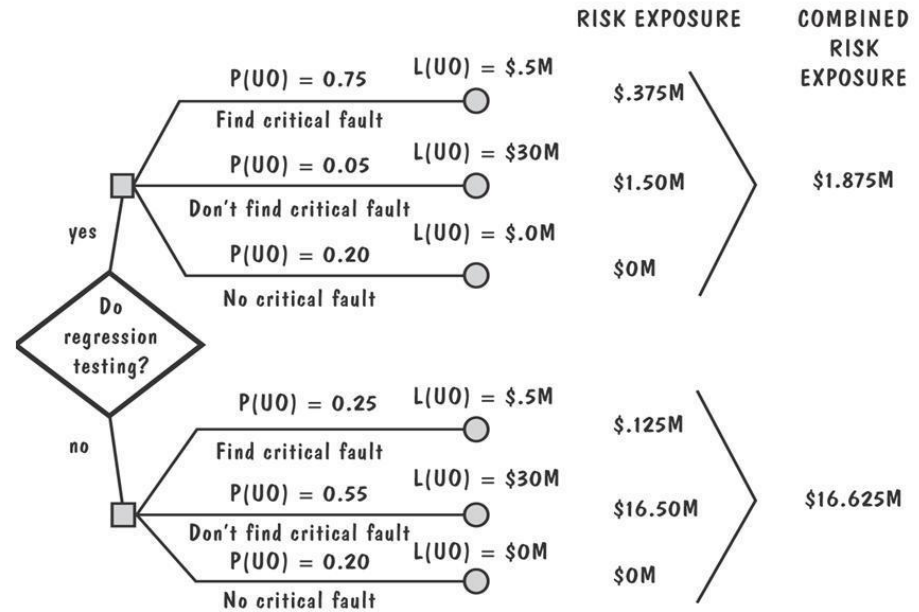
3.4 Risk Management

Risk Management Activities

- Example of risk exposure calculation

PU: prob. of unwanted outcome

LU: lost assoc with unwanted outcome



3.4 Risk Management

Risk Management Activities

- Three strategies for risk reduction:
 - *Avoiding the risk:* change requirements for **performance or functionality**
 - *Transferring the risk:* transfer to **other system, or buy insurance**
 - *Assuming the risk:* accept and control it
- Cost of reducing risk
 - *Risk leverage = (risk exposure before reduction – (risk exposure after reduction) / (cost of risk reduction)*

3.4 Risk Management

Sidebar 3.4 Boehm's Top Ten Risk Items

- Personnel shortfalls
- Unrealistic schedules and budgets
- Developing the wrong functions
- Developing the wrong user interfaces
- Gold-plating
- Continuing stream of requirements changes
- Shortfalls in externally-performed tasks
- Shortfalls in externally-furnished components
- Real-time performance shortfalls
- Straining computer science capabilities

3.5 Project Plan

Project Plan Contents(15)

- **Project scope**
- **Project schedule**
- **Project team organization**
- **Technical description of system**
- **Project standards and procedures**
- **Quality assurance plan**
- **Configuration management plan**
- **Documentation plan**
- **Data management plan**
- **Resource management plan**
- **Test plan**
- **Training plan**
- **Security plan**
- **Risk management plan**
- **Maintenance plan**

3.5 Project Plan

Project Plan Lists(3)

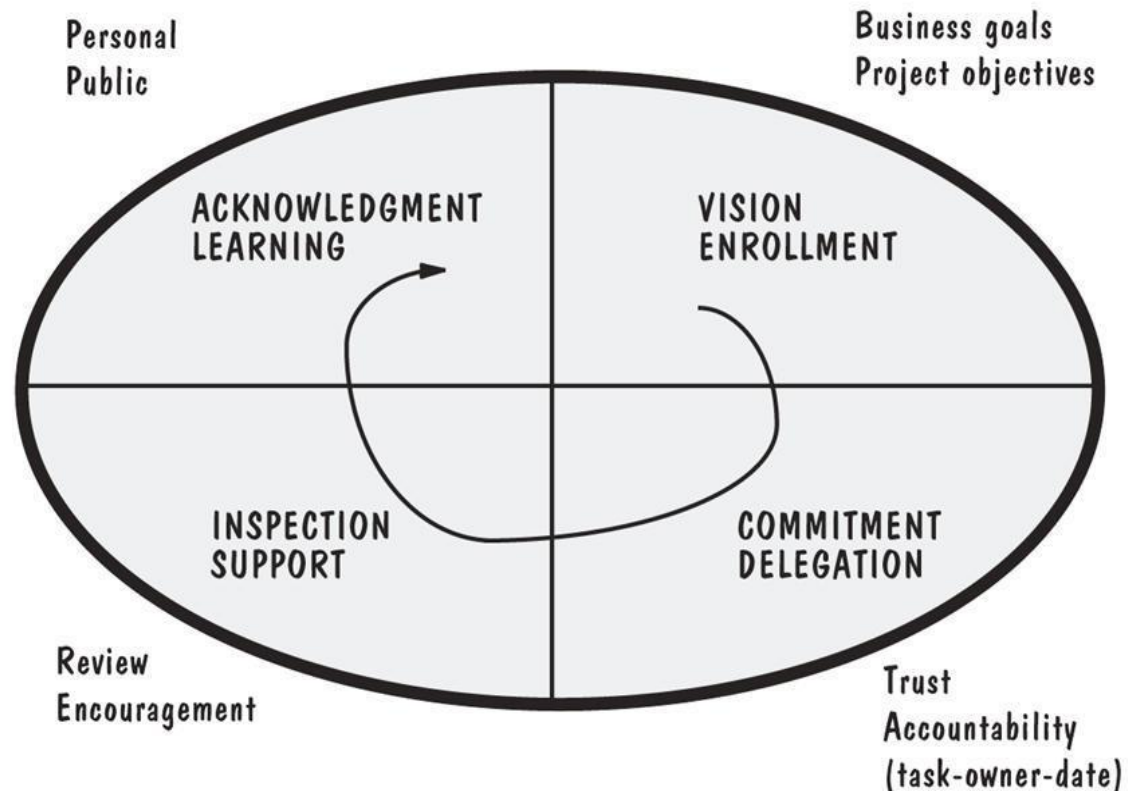
- List of the people in development team.
- List of hardware and software.
- Standards and methods, such as
 - algorithms
 - tools
 - review or inspection techniques
 - design language or representations
 - coding languages
 - testing techniques

3.6 Process Models and Project Management Enrollment Management Model: Digital Alpha AXP

- Establish an appropriately large shared vision.
- Delegate completely and elicit specific commitments from participants.
- Inspect vigorously and provide supportive feedback.
- Acknowledge every advance and learn as the program progresses.

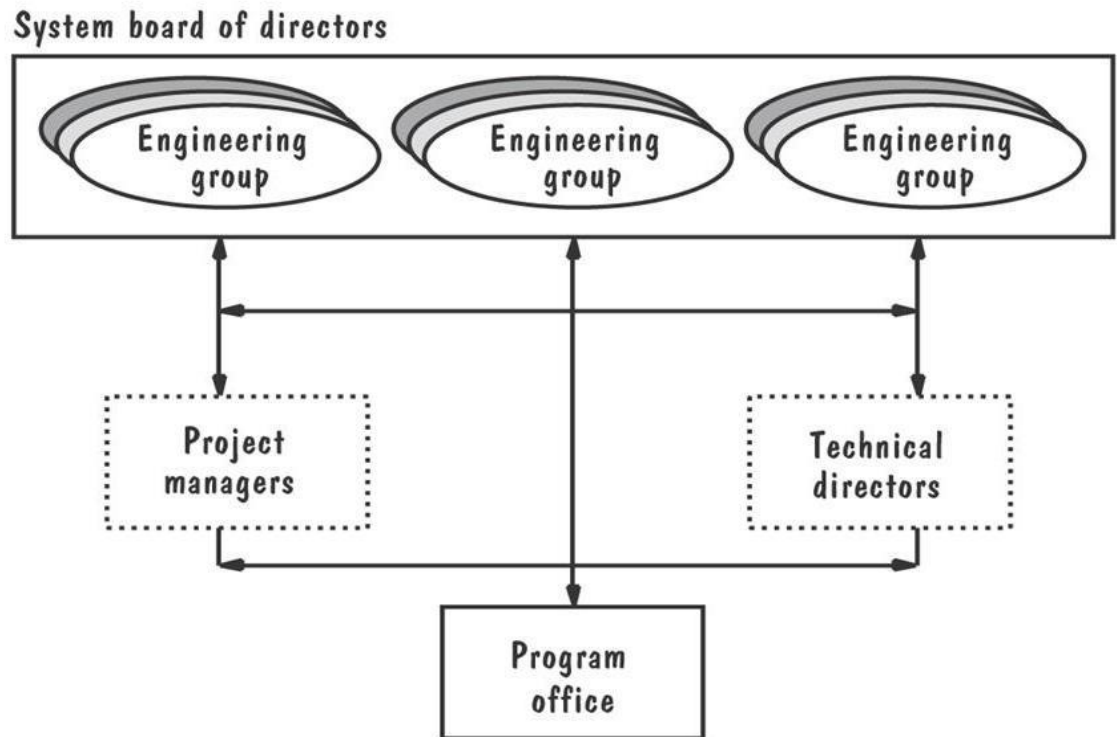
3.6 Process Models and Project Management Digital Alpha AXP

- **Vision:** To “enroll” the related programs, so they all shared common goals.



3.6 Process Models and Project Management Digital Alpha AXP

- An organization that allowed technical focus and project focus to contribute to the overall program.

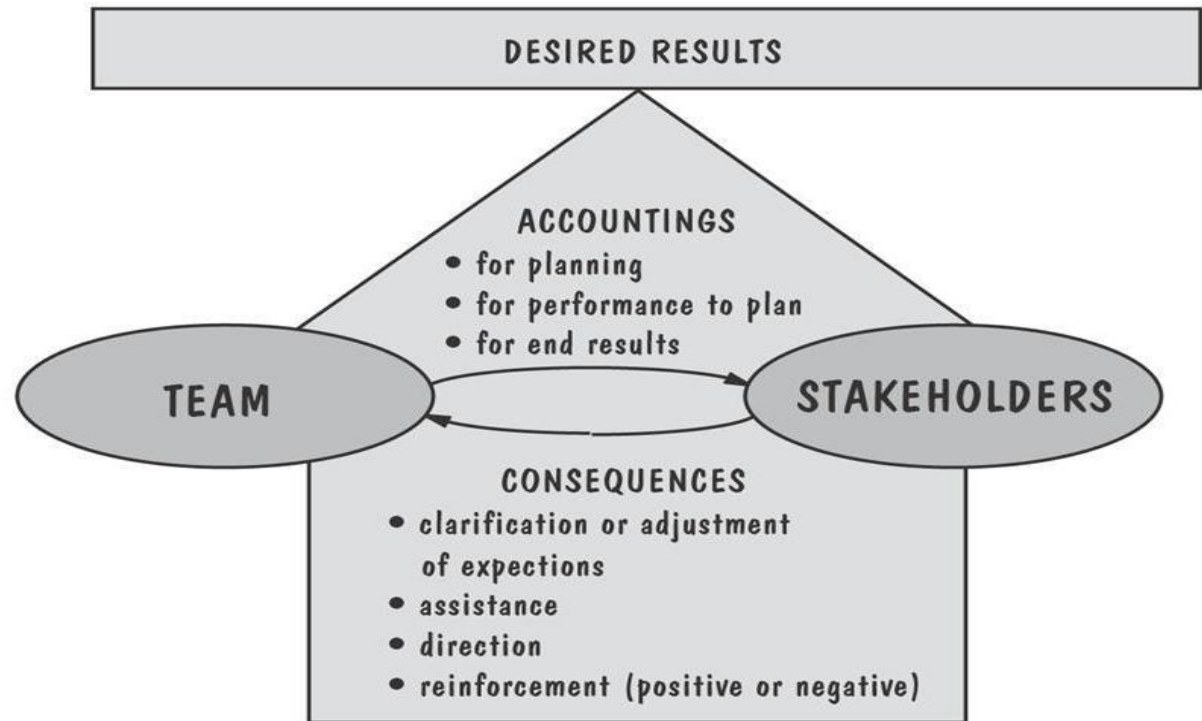


3.6 Process Models and Project Management Accountability modeling: Lockheed Martin

- **Matrix organization**
 - Each engineer belongs to **a functional unit based** on type of skill.
- **Integrated product development team**
 - Combines people from **different functional units** into interdisciplinary work unit.
- **Each activity tracked using cost estimation, critical path analysis, schedule tracking**
 - Earned value a **common measure** for progress.

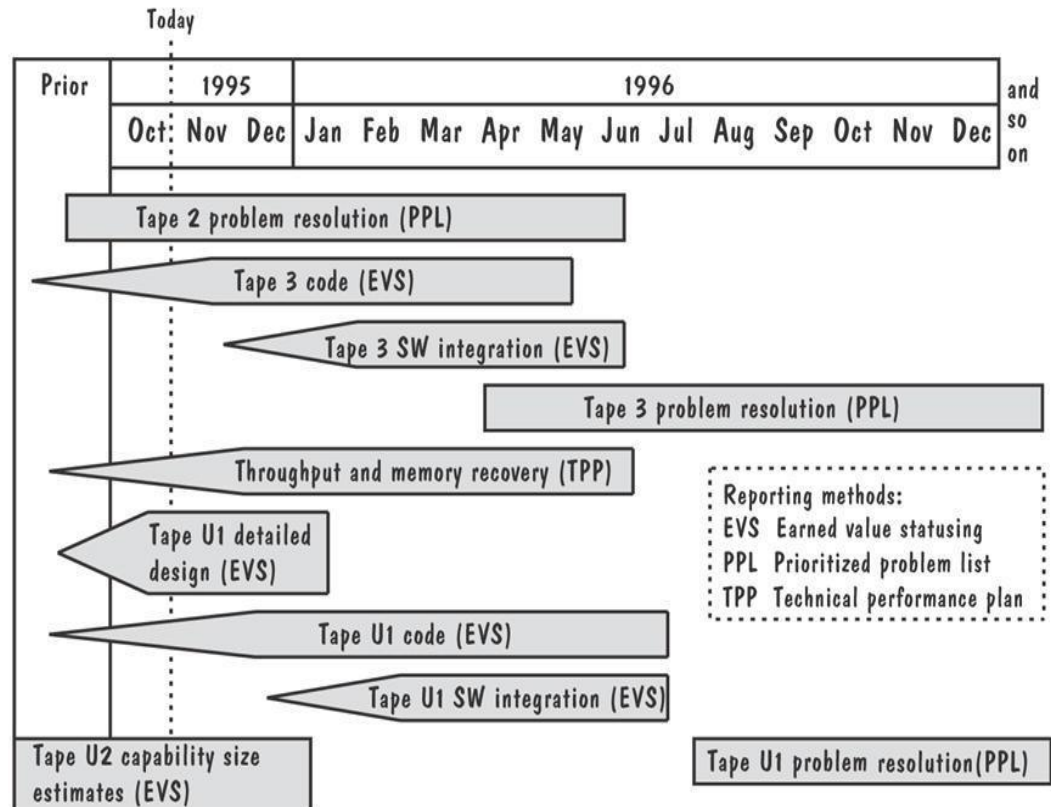
3.6 Process Models and Project Management Accountability modeling: Lockheed Martin

- Accountability model used in F-16 Project



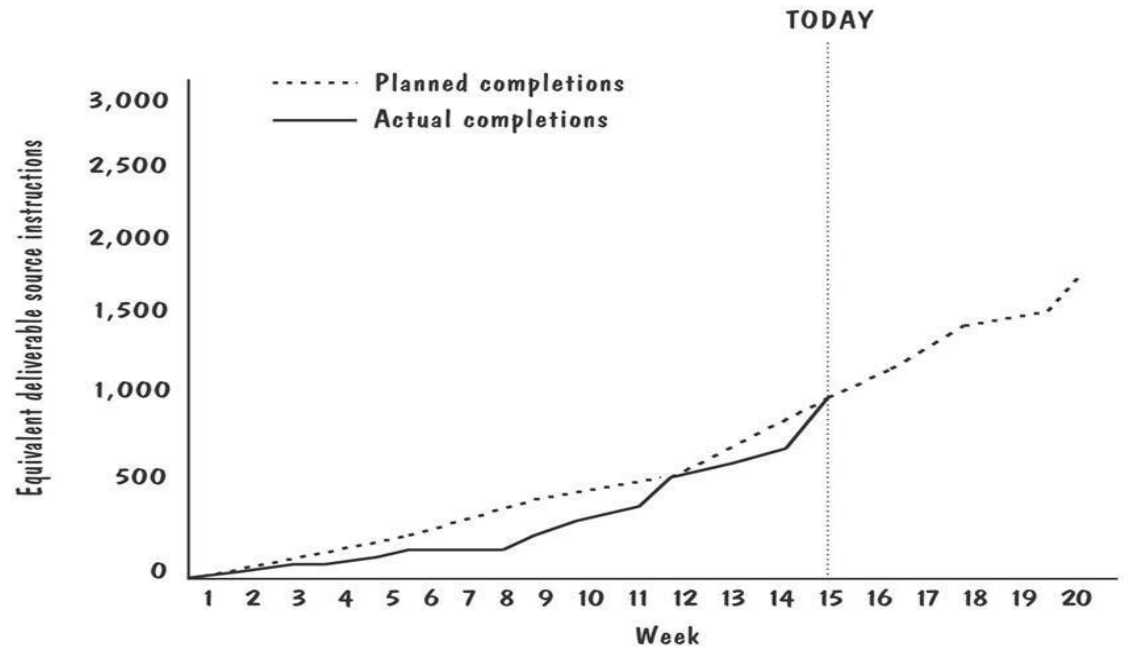
3.6 Process Models and Project Management Accountability modeling: Lockheed Martin

- Teams had multiple, overlapping activities.
- An activity map used to illustrate progress on each activity.



3.6 Process Models and Project Management Accountability modeling: Lockheed Martin

- Each activity's progress was tracked using earned value chart.

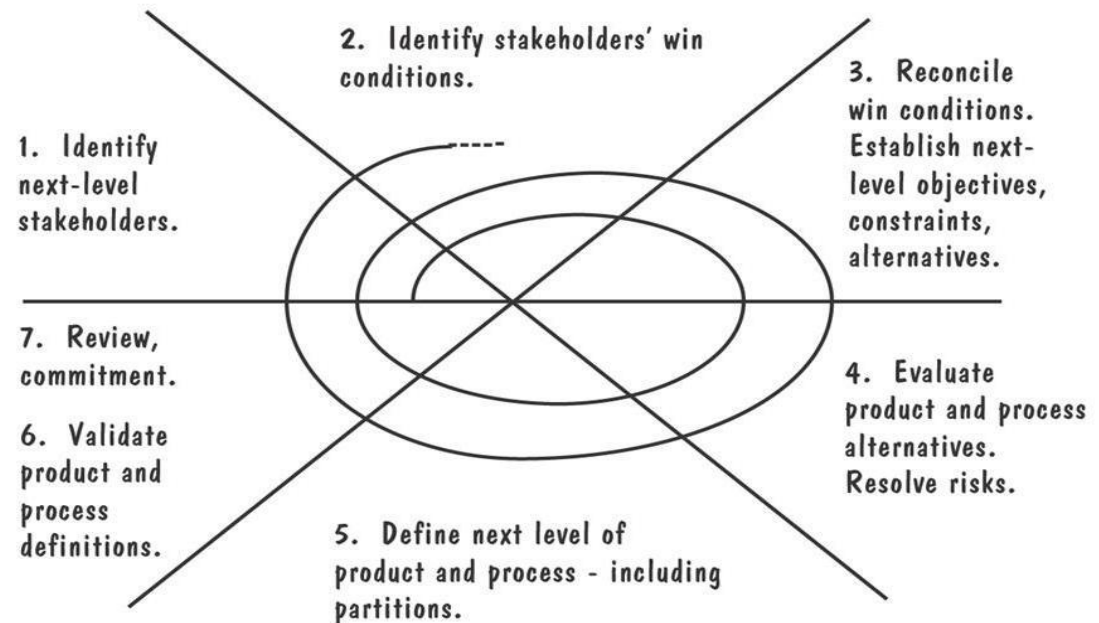


3.6 Process Models and Project Management Anchoring (Common) Milestones

- **Life cycle objectives:**
 - **Objectives:** Why is the system being developed?
 - **Milestones and schedules:** What will be done by when?
 - **Responsibilities:** Who is responsible for a function?
 - **Approach:** How will the job be done, technically and managerially?
 - **Resources:** How much of each resource is needed?
 - **Feasibility:** Can this be done, and is there a good business reason for doing it?
- **Life-cycle architecture:** define the system and software architectures and address architectural choices and risks.
- **Initial operational capability:** readiness of software, deployment site, user training.

3.6 Process Models and Project Management Anchoring milestones

- The Win-Win spiral model suggested by Boehm is used as supplement to the milestones.



3.7 Information System Example

Piccadilly System

- Using COCOMO II
- Three screens and one report
 - **Booking screen**: complexity simple, weight 1
 - **Ratecard screen**: complexity simple, weight 1
 - **Availability screen**: complexity medium, weight 2
 - **Sales report**: complexity medium, weight 5
- Estimated effort = 182 person-month

3.8 Real Time Example

Ariane-5 System

- The [Ariane-5 destruction](#) might have been prevented had the project managers developed a [risk management plan](#).
 - **Risk identification**: possible problem with reuse of the Ariane-4).
 - **Risk exposure**: prioritization would have identified if the inertial reference system (SRI) did not work as planned.
 - **Risk control**: assesment of the risk using reuse software.
 - **Risk avoidance**: using SRI with two different designs.

3.7 What this Chapter Means for You

- Key concepts in project management
 - Project planning
 - Cost and schedule estimation
 - Risk management
 - Team Organization
- Project planning involves input from all team members
- Communication path grows as the size of the team increases and need to be taken into account when planning and estimating schedule

4.3 COCOMO Algorithmic Cost Model

COCOMO (COⁿstructive CO^st MO^del) is an algorithmic cost estimation technique proposed by Boehm, which works in a bottom-up manner.¹⁵ It is designed to provide some mathematical equations to estimate software projects. These mathematical equations are based on historical data and use project size in the form of KLOC.

The COCOMO model uses a *multivariable* size estimation model for effort estimation. A multivariable model depends on several variables, such as development environment, user involvement, memory constraints, technique used, etc. A *single variable* model is based only upon the size of the project, which is given as follows:¹⁶

$$\text{Effort} = a \times \text{Size}^b$$

Size is measured in terms of KLOC (thousand delivered lines of code) per person-month (PM). The constants a and b are derived from the historical data of the past projects in the organizations.¹⁷ The values of a and b in the COCOMO model vary across the three categories of projects: organic, semidetached, and embedded, as shown in Table 4.2.

Table 4.2 The values of a and b constants

Project category	a	b
Organic	3.2	1.05
Semidetached	3.0	1.12
Embedded	2.8	1.20

Organic projects are very simple and can be developed with a small-size team. The team should have good application experience and should be familiar with the application environments. A simple data processing system is a good example of the organic category. *Embedded projects* are very complex and have stringent constraints, for example, flight control system for aircraft. *Semidetached projects* are intermediate in size and complexity. The team should have mixed experience to meet the mix of rigid and less-than-rigid requirements. A transaction processing system with fixed requirements for terminal hardware and database software is an example of the semidetached category.

COCOMO estimation is a family of hierarchical models, which includes *basic*, *intermediate*, and *detailed* COCOMO models. Each of the models initially estimates efforts based on the total estimated KLOC. We will discuss the estimation mechanism for all these models below.

Basic COCOMO Model

The basic COCOMO model estimates effort in a function of the estimated KLOC in the proposed project. The basic COCOMO model is very simple, quick, and applicable to small to medium organic-type projects. It is given as follows:

$$\text{Development effort } (E) = a \times (\text{KLOC})^b$$

$$\text{Development time } (T) = c \times (E)^d$$

Where a , b , c , and d are constants and these values are determined from the historical data of the past projects. The values of c and d for organic-, semidetached-, and embedded-type projects are shown in Table 4.3. The development time (T) is calculated from the initial development effort (E).

Table 4.3 The values of c and d constants

Project category	c	d
Organic	2.5	0.38
Semidetached	2.5	0.35
Embedded	2.5	0.32

Example 4.2

Assume that a system for simple student registration in a course is planned to be developed and its estimated size is approximately 10,000 lines of code. The organization is proposed to pay ₹25,000 per month to software engineers. Compute the development effort, development time, and the total cost for product development.

Solution:

The project can be considered an organic project. Thus, from the basic COCOMO model,

$$\text{Development effort } (E) = 3.2 \times (10)^{1.05} = 35.90 \text{ PM}$$

$$\text{Development time } (T) = 2.5 \times (35.90)^{0.38} = 9.747 \text{ months}$$

$$\begin{aligned} \text{Total product development cost} &= \text{Development time} \times \text{Salaries of engineers} \\ &= 9.747 \times 25,000 \\ &= ₹2,43,675 \end{aligned}$$

Intermediate COCOMO Model

The basic COCOMO model determines the effort and time on the basis of project size. There are certain other factors that can affect the project size and hence development effort and time. Therefore, Boehm has introduced 15 cost drivers considering the various aspects of product development environment. These cost drivers are used to adjust the project complexity for estimation of effort and these are termed as *effort adjustment factors (EAF)*. These cost drivers are classified as computer attributes, product attributes, project attributes, and personnel attributes. The value of each cost attribute is rated by the project manager in the specified project. These values vary with the project nature and the development environment.

The cost attributes and their ratings are shown in Table 4.4.¹⁸ Here, the rating scale considered is very low, low, nominal, high, very high, and extra high. These values range from 0.9 to 1.4. The effort adjustment attributes are helpful in providing an accurate estimate in the project. The intermediate COCOMO model computes software development effort as a function of the program size and a set of cost drivers. The intermediate COCOMO model estimates the initial effort using the basic COCOMO model. Then the EAF is

Table 4.4 Cost drivers and their values

Cost drivers	Ratings					
	Very low	Low	Nominal	High	Very high	Extra high
Product attributes						
Software reliability (RELY)	0.75	0.88	1.00	1.15	1.40	–
Size of database (DATA)	–	0.94	1.00	1.08	1.16	–
Product complexity (CPLX)	0.70	0.85	1.00	1.15	1.30	1.65
Hardware attributes						
Runtime performance constraints (TIME)	–	–	1.00	1.11	1.30	1.66
Memory storage constraints (STORE)	–	–	1.00	1.06	1.21	1.56
Virtual machine volatility (VIRT)	–	0.87	1.00	1.15	1.30	–
Required turnabout time (TURN)	–	0.87	1.00	1.07	1.15	–
Personnel attributes						
Analyst capability (ACAP)	1.46	1.19	1.00	0.86	0.71	–
Applications experience (AEXP)	1.29	1.13	1.00	0.91	0.82	–
Programmer capability (PCAP)	1.42	1.17	1.00	0.86	0.70	–
Virtual machine experience (VEXP)	1.21	1.10	1.00	0.90	–	–
Programming language experience (LEXP)	1.14	1.07	1.00	0.95	–	–
Project attributes						
Modern programming practices (MODP)	1.24	1.10	1.00	0.91	0.82	–
Use of software tools (TOOL)	1.24	1.10	1.00	0.91	0.83	–
Development schedule (SCHED)	1.23	1.08	1.00	1.04	1.10	–

drivers. Finally, the total effort is determined by multiplying the initial effort with the total value of EAF. The computation steps are summarized below.

Development effort (E):

$$\text{Initial effort } (E_i) = a \times (\text{KLOC})^b$$

$$\text{EAF} = \text{EAF}_1 \times \text{EAF}_2 \times \dots \times \text{EAF}_n$$

$$\text{Total development effort } (E) = E_i \times \text{EAF}$$

$$\text{Development time } (T) = c \times (E)^d$$

Example 4.3

Suppose a library management system (LMS) is to be designed for an academic institution. From the project proposal, the following five major components are identified:

Online data entry	- 1.0 KLOC
Data update	- 2.0 KLOC
File input and output	- 1.5 KLOC
Library reports	- 2.0 KLOC
Query and search	- 0.5 KLOC

The database size and application experience are very important in this project. The use of the software tool and the main storage is highly considerable. The machine experience and its volatility can be kept low. All other cost drivers are nominal requirements. Use the COCOMO model to estimate the development effort and the development time.

Solution:

The LMS project can be considered an organic category project. The total size of modules is 7 KLOC. The development effort and development time can be calculated as follows:

Development effort

$$\begin{aligned}\text{Initial effort } (E_i) &= 3.2 \times (7)^{1.05} \\ &= 24.6889 \text{ PM}\end{aligned}$$

$$\begin{aligned}\text{EAF} &= 1.16 \times 0.82 \times 0.91 \times 1.06 \times 1.10 \times 0.9 \\ &= 0.8780\end{aligned}$$

$$\begin{aligned}\text{Total effort } (E) &= E_i \times \text{EAF} \\ &= 24.6889 \times 0.8780 \\ &= 21.6785 \text{ PM}\end{aligned}$$

$$\begin{aligned}\text{Development time } (T) &= 2.5 \times (E)^{0.38} \text{ months} \\ &= 2.5 (21.6785)^{0.38} \text{ months} \\ &= 8.0469 \text{ months}\end{aligned}$$

Detailed COCOMO Model

The detailed COCOMO model inherits all the features of the intermediate COCOMO model for the overall estimation of the project cost. For detailed planning and scheduling, it is necessary to assess the impact of the cost drivers in a phased manner. The detailed COCOMO model uses different effort multipliers (cost drivers) for each phase of the project. Phase-wise effort multipliers provide better estimates than the intermediate model. The detailed COCOMO model defines five life cycle phases for effort distribution: plan and requirement, system design, detailed design, code and unit test, and integration and test. The percentage of effort in a phase varies with the project size and the project nature. The phase-wise percentage of effort distribution for some KLOC values is shown in Table 4.5.

In the detailed COCOMO model, effort is calculated as a function of size in terms of KLOC and the value of a set of cost drivers according to each phase of the software life cycle. If the project size varies majorly from the value taken in the phase-wise distribution Table 4.5, then interpolation formula can be applied to find the more appropriate

Table 4.5 Phase-wise distribution of the development effort and time

Project type and size	Plan and requirement	System design	Detailed design	Code and unit test	Integration and test
Percentage-wise distribution of the development effort					
Organic (2 KLOC)	6	16	26	42	16
Organic (32 KLOC)	6	16	24	38	22
Semidetached (32 KLOC)	7	17	25	33	25
Semidetached (128 KLOC)	7	17	24	31	28
Embedded (128 KLOC)	8	18	25	26	31
Embedded (320 KLOC)	8	18	24	24	34
Percentage-wise distribution of the development time					
Organic (2 KLOC)	10	19	24	39	18
Organic (32 KLOC)	12	19	21	34	26
Semidetached (32 KLOC)	20	26	21	27	26
Semidetached (128 KLOC)	22	27	19	25	29
Embedded (128 KLOC)	36	36	18	18	28
Embedded (320 KLOC)	40	38	16	16	30

percentage value. The detailed COCOMO model illustrates the importance of recognizing different levels of predictability at each phase of the development cycle.

Example 4.4

Compute the phase-wise development effort for the problem discussed in Example 4.3.

Solution:

There are five components in the organic project discussed in Example 4.3: online data entry, data update, file input and output, library reports, query and search. The estimated effort (E) is 21.6785 PM. The total size is 7 KLOC, which is between 2 KLOC and 32 KLOC. Thus, the actual percentage of effort can be calculated as follows:

$$\text{Plan and requirement (\%)} = 6 + (6 - 6) / (32 - 2) \times 7 = 6\%$$

$$\text{Effort} = 0.06 \times 21.6785 \text{ PM} = 1.30071 \text{ PM}$$

$$\text{System design} = 16 + (16 - 16) / (32 - 2) \times 7 = 16\%$$

$$\text{Effort} = 0.16 \times 21.6785 \text{ PM} = 3.46856 \text{ PM}$$

$$\text{Detailed design} = 24 + (26 - 24) / (32 - 2) \times 7 = 25\%$$

$$\text{Effort} = 0.25 \times 21.6785 \text{ PM} = 5.419625 \text{ PM}$$

$$\text{Code and unit test} = 38 + (42 - 38) / (32 - 2) \times 7 = 39\%$$

$$\text{Effort} = 0.39 \times 21.6785 \text{ PM} = 8.454615 \text{ PM}$$

$$\begin{aligned} \text{Integration and test} &= 22 + (16 - 22) / (32 - 2) \times 7 = 24\% \\ &= 0.24 \times 21.6785 \text{ PM} = 5.20284 \text{ PM} \end{aligned}$$

The basic, intermediate, and detailed COCOMO models are in the category COCOMO I and these are collectively known as COCOMO 81. The COCOMO models were developed to estimate the effort, schedule, and cost of a software project. The three variations of COCOMO I use different types of parameter estimation. These models are helpful to produce repeatable estimations. Also, it is easy to modify the input data and refine and customize formulas.

COCOMO 81 assumed that the product development would proceed in a linear fashion using conventional programming languages. It is unable to deal with exceptional conditions, such as exceptional teamwork, exceptional people involved in the estimation process, etc. Rough sizing of projects and inaccurate cost estimating will result in an inaccurate estimation.

COCOMO II

COCOMO II is an extension to the original COCOMO 81 model. Since the development of COCOMO 81, software development techniques have changed. These changes have changed the development processes from linear to spiral and one-step to incremental way for developing simple to complex projects. Programming techniques and technologies have moved from conventional to component-based software development. That is, these kinds of processes emphasized investing more effort in designing and managing the software development process that is beneficial for the development of quality software.

Thus, a new, enhanced COCOMO II model was proposed to overcome the limitations of COCOMO 81. It consists of sub-models, each one providing unique features for project planning and design process. These sub-models are called the *applications composition*, *early design*, *post-architecture*, and *reuse models*. We will briefly discuss each of these models below.

Application Composition Model

The application composition model is best suitable for prototyping projects and projects where there is extensive reuse of components. It involves prototyping efforts to resolve unexpected risks related to GUI, device interaction, performance, or technology maturity. The estimation is based on the standard estimates of the developer productivity in object points/months and the use of CASE tools to support development. An object point defines the artifacts, such as screens, reports, and 3GL modules in the application to be developed.

The effort estimation in an application is computed as follows:

$$\text{Effort (E)} = (\sum \text{AP} \times (1 - \% \text{reuse}/100)) / \text{PROD}$$

$\sum \text{AP}$ is the sum of application points in the system to be developed. PROD is the productivity of programmers from the historical data rated on some scale. The

PROD for the past projects is rated as very low = 4, low = 7, nominal = 13, high = 15, and very high = 50. Effort (E) will be measured in person-months (PM). %reuse is the percentage of reusable components that will be reused in the application.

Early Design Model

The early design model is used when requirements are available but design has not yet commenced. It involves exploration of an alternative architecture or concept of operation. The estimation formula based on simple COCOMO is given as follows:

$$\text{Effort } (E) = A \times \text{Size}^B \times \text{EAF}$$

A and B are constants. The value of A is fixed at 2.94 from a large data set. The size is taken in terms of KLOC. The value of B varies with the project size and it can range from 1.1 to 1.24, depending on the project complexity, process maturity level, development flexibility, risk resolution processes, and cohesion of the development team in an organization. EAF is the effort adjustment factor based on seven characteristics of projects and processes. The seven effort adjustment factors are product reliability and complexity (RCPX), reuse required (RUSE), platform difficulty (PDIF), personnel capability (PERS), personnel experience (PREX), schedule (SCED) and support facilities (FCIL). All the seven EAFs are multiplied and the final value is multiplied with initial effort to determine the total effort.

Post-Architecture Model

This model is used once the system architecture has been designed and sufficient information is available for the development and maintenance of the system. At this stage, requirements have been refined and validated, operational features have been finalized, most of the risks have been resolved, and a framework for the product has been designed. Estimation is determined using the estimation formula of early design model as follows:

$$\text{Effort } (E) = A \times \text{Size}^B \times \text{EAF}$$

SLOC or FP can be used for determining the project size. There are 17 instead of 7 multiplicative cost drivers, as shown in Table 4.6, used for the adjustment of initial effort. The following five factors determine the project's scaling exponent B .

- Precedentedness - New project - Rated low (4)
- Development flexibility - No client involvement - Rated very high (1)
- Architecture/risk resolution - No risk analysis carried out - Rated very low (5)
- Team cohesion - New team - Rated nominal (3)
- Process maturity - Process in control - Rated nominal (3)

The value of these factors is added (for example, here it is 16) and their value in percentage (i.e., .16) is added to 1.01 to get a value of 1.17.

Table 4.6 Multiplicative cost drivers

Cost drivers	Attributes	Category
Required system reliability	RELY	Product
Product complexity	CPLX	Product
Required level of documentation	DOCU	Product
Database size	DATA	Product
Percentage of reusable components	RUSE	Product
Execution time constraint	TIME	Computer
Development platform volatility	PVOL	Computer
Memory constraints	STOR	Computer
Analysts capability	ACAP	Personnel
Personnel continuity	PCON	Personnel
Programmer capability	PCAP	Personnel
Programmer experience in application	PEXP	Personnel
Analyst experience in project domain	AEXP	Personnel
Language and tool experience	LTEX	Personnel
Use of software tools	TOOL	Project
Development schedule	SCED	Project
Multisite working	SITE	Project

Reuse Model

The reuse model focuses on system development with reuse. It helps to monitor the progress of reuse-based development and guides to the investment for producing reusable assets and reusing them in development. The cost of system development with reuse involves the cost incurred in design, implementation, generalization, testing, documentation, library support, and integration of reusable assets. The level of reuse in a system is defined as the total size of the system divided by the size of the reusable components in the system. The level of reuse can be used to monitor reuse and to estimate the cost, time, quality, and productivity.

The cost of system development can be assessed by the cost of system development with reuse and without reuse. Suppose C_s is the cost of system development with reuse, C_r and C_{rw} is the relative cost of the reused software of cost C_s in it. The relative cost of reused software and without reuse becomes 1 (i.e., $C_{rw} = 1$). If the amount of reuse in software is R , then the software without reuse in a system can be represented as $1 - R$. The relative cost of reusing a software component is generally 0.10 - 0.25.²² The cost of software can be measured as follows:²³

$$\begin{aligned} \text{Total cost } (C_s) &= \text{Cost of the reused software} + \text{Cost of the software without reuse} \\ &= (\text{Relative cost of the reused software} \times \text{reused software}) \end{aligned}$$

$$\begin{aligned}
 &+ (\text{Relative cost of the software without reuse} \\
 &\times \text{Software without reuse}) \\
 &= (C_R \times R_s) + (C_{RW} \times (1 - R_s)) \\
 &= (C_R \times R_s) + (1 \times (1 - R_s)) \\
 &= 1 + R_s (C_R - 1)
 \end{aligned}$$

→ prob with models METH, CPLX in their dependence (S) on size as a key Variable.

→ Boehm's Constructive Cost Model (COCOMO)

→ Stage 1: builds prototypes, little is known abt the likely size of final product.

Stage 2: design stage - function pts as a size measure.

Stage 3: post architecture, lines of code & many cost factors can be estimated with some degree of comfort.

$$E = b S^c m(X)$$

$$FP = \text{count-total} \times (0.65 + 0.01 \times \sum F_i)$$

Initial size-based estimate $b S^c$ is adjusted by the vector of cost drivers information.

$m(X)$ - cost drivers ^{information} at each stage.

→ Cost drivers rating

extra low, very low, low, nominal, high, very high, extra high.

MLC Learning Methods

- Also called as Case-Based Reasoning (CBR)

→ Estimation using CBR involves 4 steps

1. The user identifies a new prob
2. The sys retrieves similar cases from a repository of historical info.
3. The sys reuses knowledge from previous cases.
4. The sys suggests a soln for the new case.

Basel COCOMO model

Simple, quick & applicable to small to medium organic type projects.

$$E = a \times (\text{KLOC})^b$$

$$T = c \times (E)^d$$

→ estimates effort in a function of estimated KLOC in proposed project.

	a	b	c	d
organic	3.2	1.05	2.5	0.38
Semidetached	3.0	1.12	2.5	0.35
Embedded	2.8	1.20	2.5	0.32

Intermediate COCOMO model

Determines effort & time on basis of project size.

Effort adjustment factors (EAF)

$$E = a \times (kLOC)^b$$

$$EAF = EAF_1 \times EAF_2 \times \dots \times EAF_n$$

$$E = E_1 \times EAF$$

$$T = C \times (E)^d$$

→ There are certain other factors that can affect the project size & hence the development effort & time.

→ Boehm introduced 15 cost drivers. These cost drivers are used to adjust the project complexity for estimation of effort & there are termed as EAF.

eg: cost drivers & their values.

Cost drivers

Very low low Nominal High V.H. E.H.

Product attributes

slw reliability (RELY)

0.75

0.88

1.00

1.15

1.40

-

Size of db (DATA)

-

0.94

1.00

1.08

1.16

-

product complexity (CPLX)

0.70

0.85

1.00

1.15

1.30

1.65

Detailed COCOMO model

②

- Inherits all the features of the Intermediate COCOMO model for the overall estimation of the project cost.
- In this model cost drivers are used in a phased manner. It uses different multipliers (cost drivers) for each phase of the project.
- It defines five life cycle phases for effort estimation. Plan & req., Sys. design, detailed design, code & unit test, Integration & test. The % of effort in a phase varies with proj. size & proj. nature.
- The above three models comes in the category of COCOMO I & are collectively known as COCOMO 81.

COCOMO II: It is an extension to the original COCOMO 81 model. The sub-models are called.

- > applications composition
- > early design
- > post architecture
- > Reuse models.

Application composition model: Is best suitable for prototyping projects where there is extensive reuse of components.

→ It involves prototyping efforts to resolve unexpected risks related to GUI, device interaction.

Performance or technology maturity.

AP - Sum of application pks in the system to be developed.

$$\text{Effort } E = \sum AP - (1 - \% \text{ reuse} / 100) / \text{PROD}$$

PROD - Productivity of Programmers from Historical data.

Early Design Model

$$\text{Cost} = \text{Total} \times IL = 4, L = 7, N = 13, H = 15, VH = 50$$

It is used when req's are available but design has not yet commenced.

$$E = A \times \text{Size}^B \times EAF$$

A, B are constants. A = 2.94 for large dataset
B = 1.1 to 1.24 depending on Proj. complexity, flexibility etc.

Post-Architecture model:

→ Req's have been refined & validated, risks have been ~~not~~ resolved, framework of product has been designed.

$$E = A \times \text{Size}^B \times EAF$$

SLOC or FP can be used for determining the project size. Here 17 instead of 7 multiplicative cost drivers.

17 cost drivers

Attributes.

Req. System Reliability

RELY

Product. complexity

CPLX

Req. level of documentation

DOCU

Database size

DATA

Reuse model

(3)

→ focus on system development with reuse.

Total cost (C_s) = Cost of reused slw + cost of slw

= (Relative cost of reused slw ^{without reuse} × Reused slw)
+ (Relative cost of the slw without reuse ×
slw without reuse)

$$= (C_R \times R_s) + (C_{RW} \times (1 - R_s))$$

$$= (C_R \times R_s) + (1 \times (1 - R_s))$$

$$= 1 + R_s (C_R - 1)$$
